# BIG-IP Advanced WAF

## CSV Upload Protection: Lab Build Guide

F5 Solutions Engineering

Version 1.0 • 2026-10

Customer lab guide. For evaluation and testing, not a production design.

# Contents

**Introduction** **3**
What the lab demonstrates . . . 3
Architecture . . . 3
The components . . . 4
What this lab is not . . . 4
How the guide is organised . . . 5

**Planning and prerequisites** **6**
Planning worksheet . . . 6
Worked example . . . 6
BIG-IP . . . 7
Lab VMs . . . 7
Connectivity . . . 8
The lab bundle . . . 8

**Prepare the two Linux VMs** **9**
Run the preparation script . . . 9
What the script does, and why . . . 9
Check connectivity from the BIG-IP . . . 10
Common problems . . . 10

**Build the lab services** **11**
Start the validator and the upload application . . . 11
Start the ICAP antivirus . . . 11
Generate the test files and a certificate . . . 12
Try the validator on its own . . . 12

**BIG-IP building blocks** **13**
Check the BIG-IP is ready . . . 13
Partition . . . 13
Routes to the lab hosts . . . 14
Certificate and key . . . 14
Client SSL profile . . . 15
HTTP profile . . . 16
Health monitors . . . 17
Pools . . . 18

**The Advanced WAF policy** **21**
Create the policy . . . 21
General settings . . . 22
Allowed URL . . . 23
Allowed file type . . . 24
Parameters . . . 25
Learning and blocking settings . . . 28
Blocking response page . . . 29
Apply the policy . . . 29

**Antivirus scanning** **30**
Point the BIG-IP at the scanner . . . 30
Restart the Application Security service . . . 30
Turn scanning on in the policy . . . 31

**The validator hop** **32**
ICAP profile . . . 32
Internal virtual server . . . 32
Request-adapt profile . . . 33
Inner virtual server . . . 34
Why there are two virtual servers . . . 36

**The entry point** **37**
LTM policy . . . 37
Outer virtual server . . . 38
The first uploads . . . 39

**Acceptance tests** **40**
Run the tests . . . 40
What is tested . . . 40
Outage tests . . . 41
Acceptance criteria . . . 41

**Operations** **42**
Finding out what happened to an upload . . . 42
Troubleshooting . . . 42
Things to try . . . 43
Removing the lab . . . 43

**Show the lab to others** **44**
1. A good file goes through . . . 44
2. The WAF stops what is not an upload . . . 44
3. The antivirus stops a valid CSV that carries a payload . . . 44
4. The validator judges what the WAF cannot . . . 44
5. Nothing gets in behind the BIG-IP . . . 45
6. Every service fails closed . . . 45
7. Where the evidence is . . . 45
What to say it is not . . . 45

**Appendix A: scripted build** **46**

**Appendix B: object inventory** **47**

**Appendix C: what the validator checks** **48**
The formula rule . . . 48

**Appendix D: things to know about BIG-IP 21.1.0.2** **50**

**Appendix E: references** **51**

# Introduction

This guide walks you through building a small lab that shows how BIG-IP Advanced WAF can protect an application that accepts CSV file uploads. You build it in your own environment, run the supplied tests against it, and keep it for as long as you want to experiment.

It is written for the engineer who builds the lab. It assumes working knowledge of BIG-IP Local Traffic Manager (LTM) objects and a Linux host that can run containers. No prior Advanced WAF experience is needed: each task says why it is there before it says what to click.

Every BIG-IP setting in this guide was built and tested end to end on BIG-IP Virtual Edition 21.1.0.2, Classic BIG-IP (TMOS). Menu paths refer to that release. This guide does not apply to F5OS or BIG-IP Next.

## What the lab demonstrates

- **Request protection at the BIG-IP.** Only `POST /upload` with one file is accepted. Wrong methods, wrong content types, oversized requests, extra form fields, files that are not named `.csv` and executables renamed to `.csv` are blocked before anything else sees them.
- **Malware scanning before delivery.** The BIG-IP sends each uploaded file to an antivirus scanner over ICAP and blocks the upload if the scanner finds something, or cannot be reached.
- **CSV validation as a service.** A small external service decides whether the file really is a well-formed CSV and whether any cell would run as a spreadsheet formula. The BIG-IP asks it for a verdict; it never touches the application.
- **Fail closed at every step.** If any service is down, the upload is not delivered.
- **Traceability.** A blocked request carries a BIG-IP support ID. A validated one carries an upload ID and a file hash that appear in every log.

## Architecture

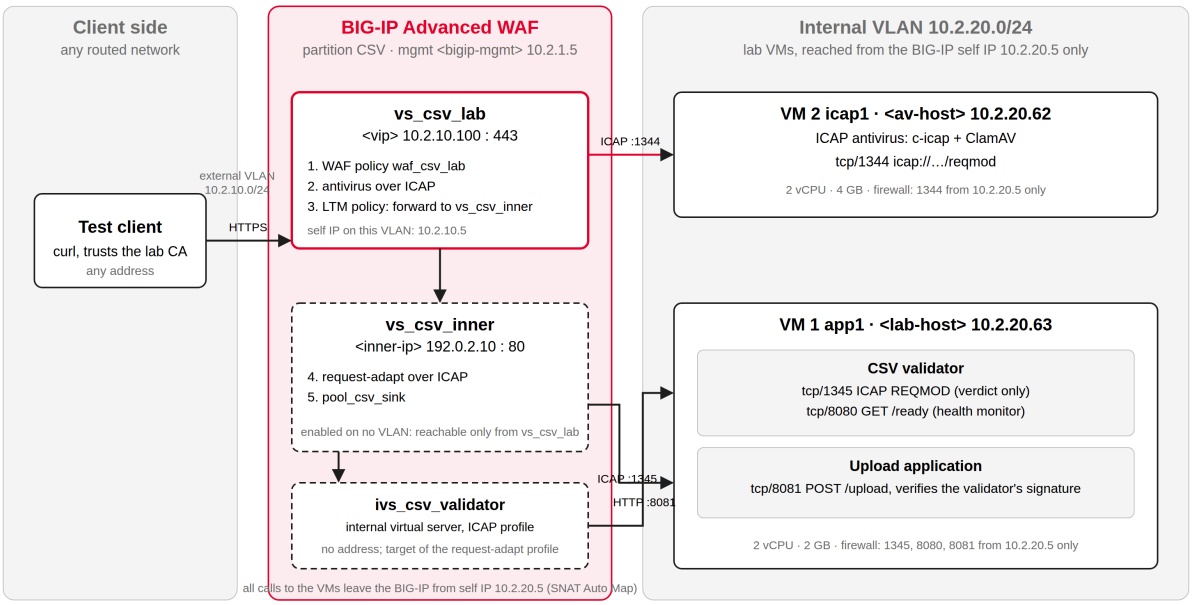



**Figure 1:** The lab, with the addresses used in this guide's worked example. Your values go in the planning worksheet in chapter 2.

```
test client
    | HTTPS
    v
[ BIG-IP ]  outer virtual server  <vip>:443
    |   1. Advanced WAF policy            -> block, with a support ID
    |   2. antivirus over ICAP -----------> ICAP antivirus   <av-host>:1344   (VM 2)
    |   3. forward to the inner virtual server
    v
[ BIG-IP ]  inner virtual server  (on no VLAN)
    |   4. request-adapt over ICAP -------> CSV validator    <lab-host>:1345   (VM 1)
    |        approved: request continues with signed headers
    |        rejected: the validator's error goes back to the client
    v
    pool --------------------------------> upload application <lab-host>:8081   (VM 1)
```

The BIG-IP makes every call. The three services never talk to each other. They run on two VMs: the validator and the application share one, and the antivirus has its own, as it would in a real deployment.

## The components

| Component | What it is | Role in the lab |
|---|---|---|
| BIG-IP Advanced WAF | F5 product | Request-level controls, the antivirus call, the validator call, load balancing to the application, logging |
| ICAP antivirus | Sample container: c-icap with the ClamAV engine | Scans each uploaded file |
| CSV validator | Sample container, Python | Checks the multipart structure, the file name, UTF-8, CSV grammar and the formula policy; returns a verdict |
| Upload application | Sample container, Python | Stands in for your application. Stores what it receives and lists it, so the tests can prove what did and did not arrive |

## What this lab is not

- **Not a production design.** It has had no load, high-availability, upgrade or independent security testing.
- **The three containers are sample code.** They are provided so that you can test the BIG-IP configuration. They are not F5 products and are not covered by F5 Support. In a real deployment you would use your own antivirus product and your own application, and decide whether CSV validation belongs in a service like this one or in the application.
- **Not a guarantee that a file is safe to open.** The validator enforces the documented rules in Appendix C and nothing more.
- **Not the only way.** BIG-IP can also be configured without the validator hop, or with the antivirus alone. The lab shows the fullest arrangement so that you can remove parts and see what changes.

## How the guide is organised

| Chapter | You do | Time |
|---|---|---|
| 2 | Fill in the planning worksheet and check prerequisites | 20 min |
| 3 | Prepare the two Linux VMs | 30 min |
| 4 | Start the three lab services | 20 min |
| 5 | Create the shared BIG-IP objects | 20 min |
| 6 | Build the Advanced WAF policy | 30 min |
| 7 | Turn on antivirus scanning | 10 min |
| 8 | Build the validator hop | 15 min |
| 9 | Build the entry point and send the first upload | 15 min |
| 10 | Run the acceptance tests | 15 min |
| 11 | Operate, troubleshoot and remove the lab | as needed |
| 12 | Show the lab to others | 20 min |

If you would rather script the BIG-IP side, Appendix A builds chapters 5 to 9 with one command. Building it by hand once is the better way to learn what each object does.

# Planning and prerequisites

## Planning worksheet

Fill this in before you start. The later chapters use these placeholders. The examples are the values used when this guide was tested; the screenshots show them too.

| Placeholder | Meaning | Example | Your value |
|---|---|---|---|
| `<bigip-mgmt>` | BIG-IP management address | `10.2.1.5` | |
| `<vip>` | Address clients upload to. A free address on a BIG-IP VLAN that your test client can reach | `10.2.10.100` | |
| `<lab-host>` | VM 1: the validator and the upload application | `10.2.20.63` | |
| `<av-host>` | VM 2: the ICAP antivirus | `10.2.20.62` | |
| `<bigip-self-ip>` | The BIG-IP self IP that reaches the two VMs. The VMs' firewalls admit only this address | `10.2.20.5` | |
| `<gateway>` | Next hop from the BIG-IP towards `<lab-host>` or `<av-host>`, if either is not on a BIG-IP VLAN | `10.2.20.1` | |
| `<inner-ip>` | Address of the inner virtual server. It is never on a network; any unused address works | `192.0.2.10` | |

The lab uses one administrative partition, `CSV`, so that everything it creates is easy to find and remove. Object names in this guide are fixed; keep them, because the scripts and tests expect them.

## Worked example

The examples in the worksheet are one consistent deployment, the one in the diagram in chapter 1 and in every screenshot. Reading it as a whole makes the placeholders easier to map onto your own network.

| Network | Address | Role |
|---|---|---|
| External VLAN `10.2.10.0/24` | `10.2.10.5` | BIG-IP self IP |
| | `10.2.10.100` | `<vip>`, the upload address clients use |
| Internal VLAN `10.2.20.0/24` | `10.2.20.5` | BIG-IP self IP; the source of every call to the VMs |
| | `10.2.20.63` | VM 1, `<lab-host>`: validator (1345, 8080) and application (8081) |

| Network | Address | Role |
|---|---|---|
| | `10.2.20.62` | VM 2, `<av-host>`: antivirus (1344) |
| Management | `10.2.1.5` | `<bigip-mgmt>`, Configuration utility only |
| None | `192.0.2.10` | `<inner-ip>`: the inner virtual server. A documentation address; it is never on a wire |

Two things carry over to any layout. The VMs must be reachable from a BIG-IP *self IP* (here `10.2.20.5`), not from the management address, because the calls come from the traffic path. And the VMs' firewalls admit the lab ports from that self IP only, so the WAF cannot be bypassed by reaching a service directly. If your VMs sit on a network the BIG-IP has no self IP on, the BIG-IP needs a route to them (chapter 5) and the firewall must admit the self IP the route uses.

## BIG-IP

| Requirement | Detail |
|---|---|
| Version | BIG-IP Virtual Edition 21.1.0.2. Other 21.1 builds should behave the same; earlier releases have the same features but some menu paths and field names differ |
| Licence | Advanced WAF (in the Best bundle, or licensed on its own) |
| Provisioning | **Local Traffic (LTM)** and **Application Security (ASM)** at *Nominal* |
| Resources | 4 vCPU and 8 GB of memory were used |
| Access | An account with the Administrator role |
| Network | A self IP on the VLAN that holds `<vip>`, and a path from a self IP to `<lab-host>` and `<av-host>` |

> **Use a lab BIG-IP.** Chapter 7 changes one setting that applies to the whole device, and requires a restart of the Application Security service. Do not build this on a BIG-IP that protects live applications.

## Lab VMs

Two small Linux VMs. Chapter 3 prepares them.

| | VM 1, `<lab-host>` | VM 2, `<av-host>` |
|---|---|---|
| Runs | CSV validator and upload application | ICAP antivirus |
| vCPU / memory / disk | 2 / 2 GB / 20 GB | 2 / 4 GB / 20 GB (2 GB is enough in antivirus test mode) |
| Operating system | Ubuntu 24.04 or Debian 13 (tested). Rocky or Alma 9 should work and is untested | same |

| | VM 1, `<lab-host>` | VM 2, `<av-host>` |
|---|---|---|
| Container runtime | Podman with `podman-compose`, installed by the preparation script. Docker with the Compose plugin takes the same commands but was not tested | same |
| Internet access | Once, to install packages and build the images | The same, plus outbound HTTPS to `database.clamav.net` for signatures, unless you use test mode |

Both VMs must be reachable from `<bigip-self-ip>`. Put them on a VLAN the BIG-IP has a self IP on; otherwise add a route on the BIG-IP (chapter 5).

## Connectivity

Open these paths before you start. All are TCP.

| From | To | Port | Used for |
|---|---|---|---|
| Test client | `<vip>` | 443 | Uploads |
| BIG-IP self IP | `<av-host>` | 1344 | Antivirus scanning (ICAP) |
| BIG-IP self IP | `<lab-host>` | 1345 | Validator verdicts (ICAP) |
| BIG-IP self IP | `<lab-host>` | 8080 | Validator health check |
| BIG-IP self IP | `<lab-host>` | 8081 | Upload application |
| Test host | `<lab-host>` | 8081 | The test script reads what the application received |
| Your workstation | `<bigip-mgmt>` | 443 | Configuration utility |

## The lab bundle

You received `csv-upload-lab-<version>.zip` with this guide. Unpack it on `<lab-host>`. It contains:

| Path | Contents |
|---|---|
| `docker-compose.yml` | Validator and upload application |
| `icap-av/` | ICAP antivirus image and its own compose file |
| `validator/`, `sink/` | Source of the two sample services |
| `scripts/gen-secret.sh` | Creates the secret the validator and the application share |
| `test/` | Test files, a certificate generator and the test runner |
| `bigip/` | The WAF policy as JSON and the scripted build of Appendix A |

Nothing in the bundle contains a password, a key or an address from your environment. The scripts create those locally and keep them out of version control.

# Prepare the two Linux VMs

Create two VMs on your hypervisor with the sizes from chapter 2, a current Ubuntu or Debian cloud image, a static address each, and an account you can `sudo` from. Then prepare each one with the script in the bundle. The script is small and readable; what it does is listed below so that you can do it by hand instead.

## Run the preparation script

Copy the bundle to each VM. On VM 1:

```
sudo scripts/prepare-host.sh --role app --allow <bigip-self-ip>
```

On VM 2:

```
sudo scripts/prepare-host.sh --role av --allow <bigip-self-ip>
```

Add `--check` to either command to see what it would change without changing anything. `--allow` takes a comma-separated list if more than one address should reach the lab, for example a second BIG-IP or the host you run the tests from.

**You should see** each run end with `done` and the next commands to run.

## What the script does, and why

| Step | What | Why |
|---|---|---|
| Packages | `podman`, `podman-compose`, `uidmap`, `slirp4netns`, `nftables`, `git`, `curl`, `python3`, `openssl` | Rootless Podman needs the user-namespace helpers; the lab scripts need the rest |
| Account `csvlab` | A normal user with a 65,536-ID subordinate range in `/etc/subuid` and `/etc/subgid` | The containers run as this user, not as root. Without the subordinate range, Podman cannot unpack images |
| Linger | `loginctl enable-linger csvlab` | The containers keep running after you log out and come back after a reboot |
| Log driver | `log_driver = "k8s-file"` in the user's `containers.conf` | The default driver writes to the journal, which a normal user cannot read; `podman logs` would show nothing |
| Low ports | `net.ipv4.ip_unprivileged_port_start = 1024` | The ICAP ports 1344 and 1345 are below Podman's rootless default of 1024 |
| Firewall | nftables: SSH open, the lab ports open to `--allow` only, everything else dropped | The BIG-IP is the only legitimate caller of the validator, the antivirus and the application. Anything else reaching them bypasses the WAF |

## Check connectivity from the BIG-IP

The BIG-IP must reach the VMs from a self IP, not from its management address. From the BIG-IP command line:

```
ping -c 2 -I <bigip-self-ip> <lab-host>
ping -c 2 -I <bigip-self-ip> <av-host>
```

If the VMs are on a VLAN the BIG-IP is not connected to, the pings fail until the route in chapter 5 exists. If they fail with the route in place, look at the VM firewall (`sudo nft list ruleset`) and the network in between.

**You should see** replies from both addresses.

## Common problems

| Symptom | Cause | Fix |
|---|---|---|
| `podman build` fails with "potentially insufficient UIDs or GIDs" | No subordinate ID range for the account | Re-run the script; it adds a range if `useradd` did not. Then `podman system migrate` as the account |
| `podman logs` prints nothing | Log driver is journald | The script sets `k8s-file`; recreate the containers afterwards |
| Port 1344 or 1345 "permission denied" | Rootless Podman cannot bind below 1024 | The sysctl the script sets, then restart the containers |
| The BIG-IP monitor stays red though the service answers locally | Firewall, or the BIG-IP is reaching from the wrong address | `--allow` must list the self IP on the VLAN the BIG-IP uses towards the VM. Check with `nft list ruleset` and a packet capture on the VM |
| `apt` hangs on an IPv6 address | The VM resolves mirrors to IPv6 but has no IPv6 route | Set `Acquire::ForceIPv4 "true";` in `/etc/apt/apt.conf.d/99force-ipv4` |

# Build the lab services

In this chapter you start the three sample services and prove each one answers. Do this first: the BIG-IP health monitors you create later turn green only when the services are up.

## Start the validator and the upload application

On VM 1, logged in as the `csvlab` account the preparation script created, in the unpacked bundle:

```
scripts/gen-secret.sh
podman-compose up -d --build
```

`gen-secret.sh` writes a random secret to `.env`. The validator signs each approval with it and the application checks the signature, so a request that skipped the validator is refused. Both services refuse to start without it.

**You should see** both services answer:

```
curl -s http://127.0.0.1:8080/ready     # {"status":"ready"}
curl -s http://127.0.0.1:8081/health    # {"status":"ok","service":"upload-sink"}
```

## Start the ICAP antivirus

On VM 2, as `csvlab`, in the bundle's `icap-av` directory, choose one:

```
# Full ClamAV signature database. Needs about 2 GB of memory and outbound HTTPS.
podman-compose up -d --build

# Test mode, for a small host or one with no Internet access.
CLAMAV_DB=test ICAP_AV_MEM=256m podman-compose up -d --build
```

> **Test mode is not an antivirus.** It runs the same scanner with a single signature that matches the industry-standard EICAR test string. It lets you prove that the BIG-IP calls the scanner and acts on the answer. It will not detect real malware.

In full mode the scanner also keeps that one lab signature beside the official database, named `Lab-EICAR-Anywhere` so it stands out in the logs. ClamAV's own EICAR rule only fires when the test string is the entire file, which is correct for the test file but would let the lab's "valid CSV carrying the test string" demonstration pass unnoticed. Real malware signatures match inside files; only the EICAR convention is this strict.

The first start in full mode downloads about 250 MB of signatures and can take several minutes.

**You should see** one of these lines:

```
podman logs csv-icap-av | head -3
# icap-av: FULL signature database
# icap-av: TEST signature database (EICAR only)
```

## Generate the test files and a certificate

On the host you will run the tests from. VM 1 is the natural choice: it already holds the bundle, and the test runner reads the upload application on `127.0.0.1`. If you use another host, add its address to `--allow` on VM 1 so it can read port 8081.

```
test/generate-test-files.sh
VIP_IP=<vip> test/gen-tls.sh
```

The first command writes 40 small files to `test/files/`: valid CSVs, files each layer should stop, and the EICAR test string. No real malware is included.

The second creates a private certificate authority and a server certificate whose subject alternative name is `<vip>`, under `test/tls/`. You import the certificate into the BIG-IP in chapter 5, and the test client trusts the authority, so the tests validate the certificate properly instead of ignoring it.

Copy `test/tls/server.crt` and `test/tls/server.key` to the workstation you browse the BIG-IP from.

## Try the validator on its own

The validator has a validate-only address that returns its verdict without delivering anything. It is a quick way to see the rules at work:

```
curl -s -F "file=@test/files/ok_basic.csv;type=text/csv"       http://127.0.0.1:8080/upload
# {"accepted":true,"upload_id":"lab-...","status":"validated",...}
curl -s -F "file=@test/files/bad_formula_eq.csv;type=text/csv" http://127.0.0.1:8080/upload
# {"accepted":false,...,"reason_code":"CSV_FORMULA_CELL",...}
```

`;type=text/csv` matters. Without it `curl` labels the file `application/octet-stream`, which the lab rejects. A browser sets the type by itself.

# BIG-IP building blocks

These are ordinary LTM objects. Screenshots show the *Create* form filled in with the values from the test build, with the changed fields outlined in red. Use your own worksheet values where a placeholder appears.

Log in to the Configuration utility at `https://<bigip-mgmt>/`.

## Check the BIG-IP is ready

| Screen | You should see |
|---|---|
| **System › Resource Provisioning** | *Local Traffic (LTM)* and *Application Security (ASM)* at **Nominal**. If ASM is at None, set it to Nominal and click **Submit**. The system restarts services and logs you out for about a minute |
| **System › License** | Advanced WAF listed, and a licence that has not expired |
| **Network › Self IPs** | A self IP on the VLAN that holds `<vip>` |

## Partition

A partition keeps the lab's objects together.

**System › Users › Partition List › Create**

| Field | Value |
|---|---|
| Partition Name | `CSV` |
| Description | `CSV upload validation lab` |

Click **Finished**. Then set the **Partition** selector at the top right of the page to **CSV** and leave it there for the rest of the build. If an object later turns up in *Common*, this is the step that was missed.

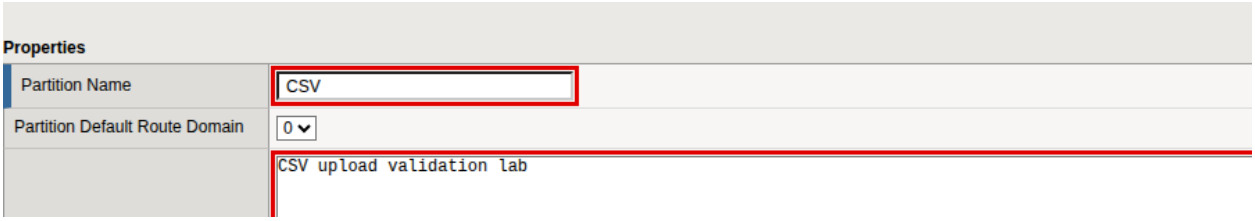



**Figure 2:** The new partition.

## Routes to the lab hosts

Skip this section if `<lab-host>` and `<av-host>` are on VLANs the BIG-IP is directly connected to. Otherwise create one route for each host that is not.

**Network › Routes › Add**

| Field | Value |
|---|---|
| Name | `csv_lab_host` (and `csv_lab_icap_av` for the antivirus host) |
| Destination | `<lab-host>` |
| Netmask | `255.255.255.255` |
| Resource | **Use Gateway…** |
| Gateway Address | `<gateway>` |

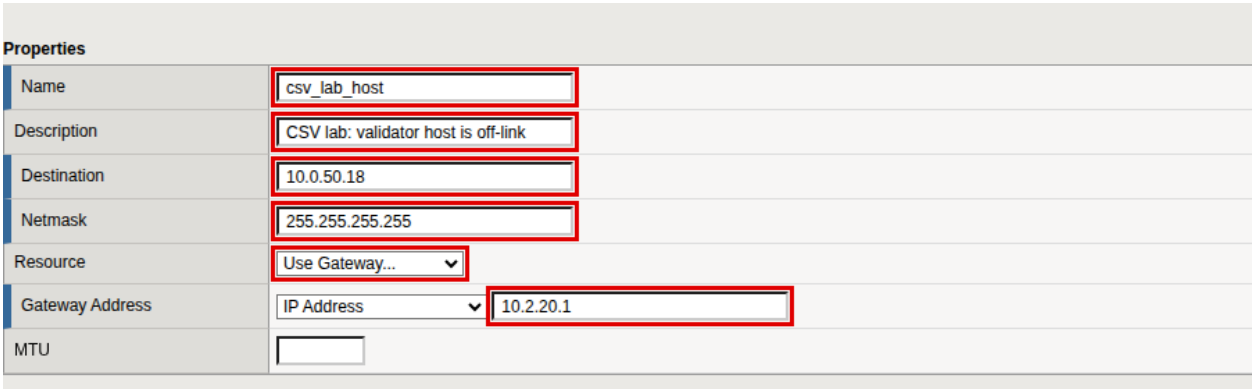



**Figure 3:** A host route to the lab host.

## Certificate and key

Import the key, then the certificate, under the same name so that the BIG-IP pairs them.

**System › Certificate Management › Traffic Certificate Management › SSL Certificate List › Import**

1. **Import Type** *Key*. **Key Name** *New*, `csv_lab`. **Key Source** *Upload File*, choose `server.key`. Click **Import**.
2. **Import** again. **Import Type** *Certificate*. **Certificate Name** *New*, `csv_lab`. **Certificate Source** *Upload File*, choose `server.crt`. Click **Import**.




**Figure 4:** Importing the key.

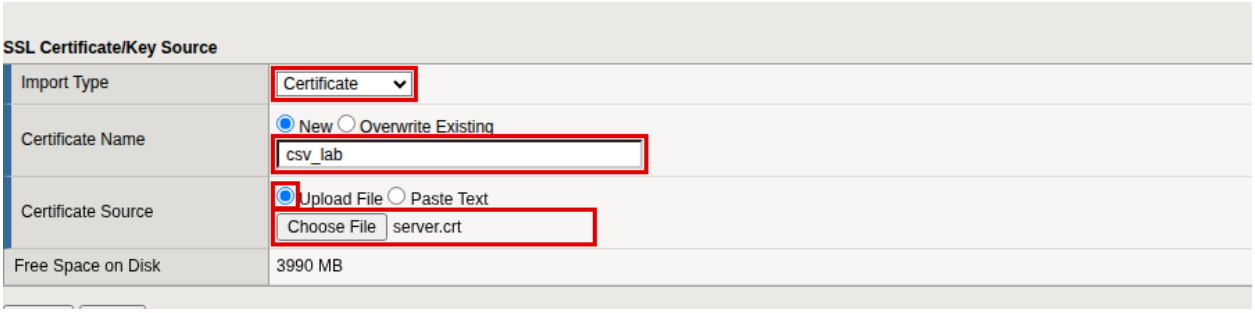



**Figure 5:** Importing the certificate under the same name.

**You should see** `csv_lab` in the list with both a certificate and a key, and `<vip>` in its subject alternative name.

## Client SSL profile

**Local Traffic › Profiles › SSL › Client › Create**

| Field | Value |
|---|---|
| Name | `clientssl_csv_lab` |
| Parent Profile | `clientssl` |
| Certificate Key Chain | Tick **Custom**, click **Add**. Certificate `csv_lab`, Key `csv_lab`, Chain *None*. Click **Add** |

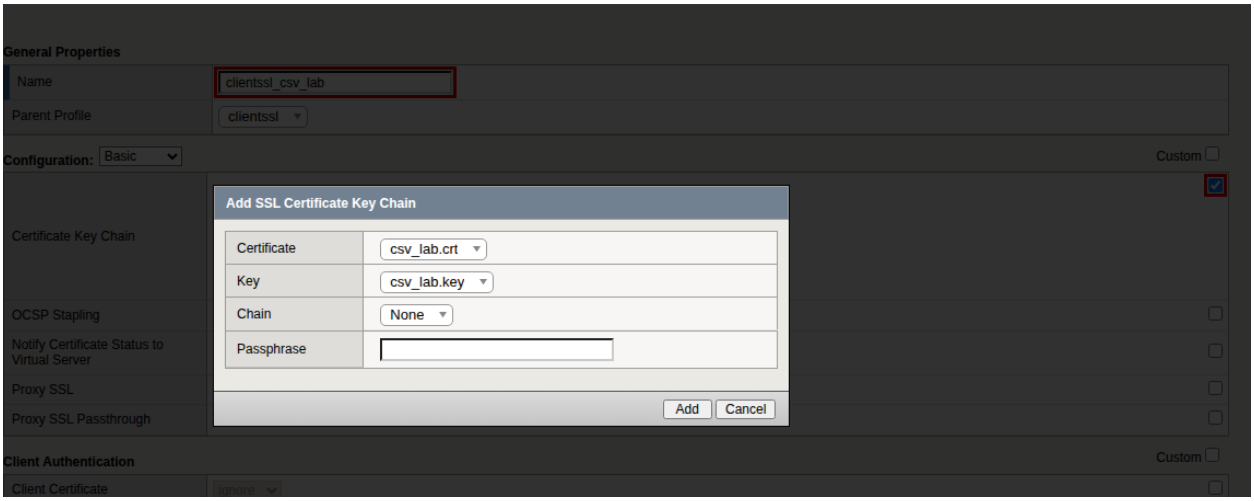



**Figure 6:** The Add SSL Certificate Key Chain dialog: certificate and key csv_lab, chain None.




**Figure 7:** The profile before Finished: parent clientssl and the chain listed.

## HTTP profile

The lab services log the real client address. This profile passes it to them in the `X-Forwarded-For` header.

**Local Traffic › Profiles › Services › HTTP › Create**

| Field | Value |
|---|---|
| Name | `http_csv_lab` |
| Parent Profile | `http` |
| Insert X-Forwarded-For | Tick **Custom**, set **Enabled** |




**Figure 8:** The HTTP profile: name and parent profile.




**Figure 9:** Insert X-Forwarded-For enabled.

## Health monitors

Two monitors, one per service. Type the `\r\n` sequences exactly as shown.

**Local Traffic › Monitors › Create**

| Field | `mon_csv_validator` | `mon_csv_sink` |
|---|---|---|
| Type | *HTTP* | *HTTP* |
| Interval / Timeout | 5 / 16 | 5 / 16 |
| Send String | `GET /ready HTTP/1.1\r\nHost: csv-validator\r\nConnection: close\r\n\r\n` | `GET /health HTTP/1.1\r\nHost: csv-sink\r\nConnection: close\r\n\r\n` |
| Receive String | `status.:.ready` | `status.:.ok` |
| Alias Service Port | `8080` | leave * |

The validator pool member is its ICAP port, which an HTTP monitor cannot speak to. The alias port points the monitor at the validator's health address instead, which reports whether the ICAP listener is serving.

The receive strings use dots where the services send double quotes (`{"status":"ready"}`). The BIG-IP drops double quotes from a receive string, so a pattern without them is used.

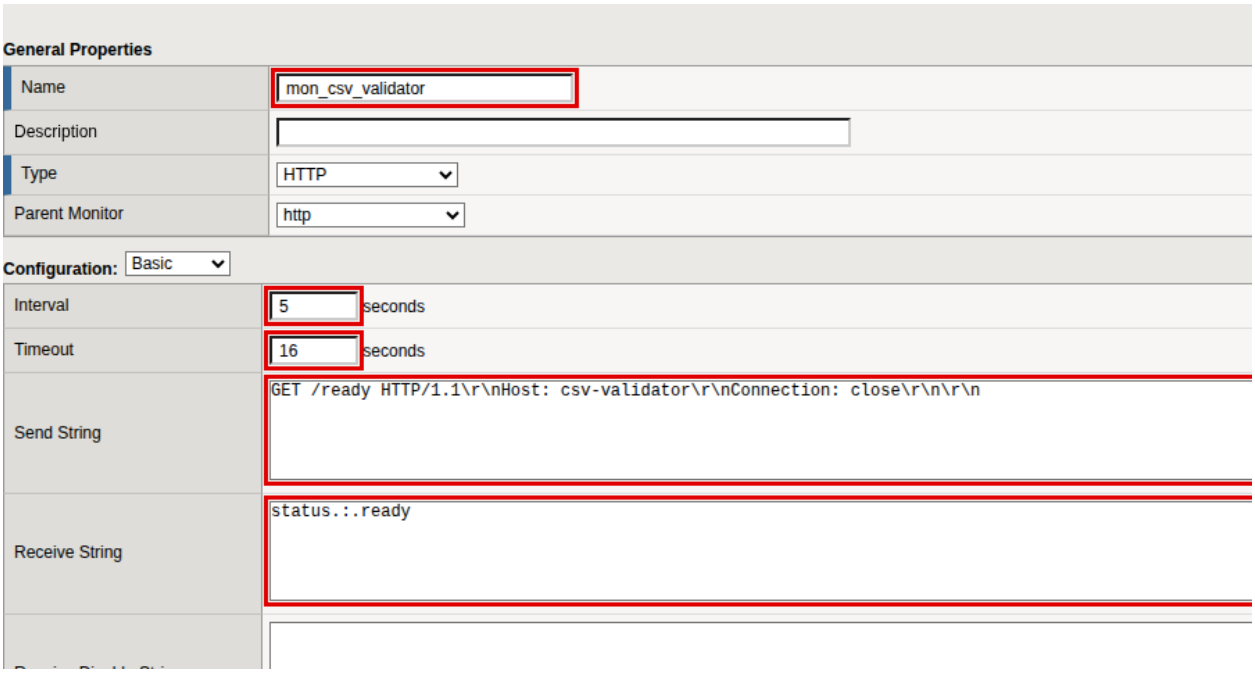



**Figure 10:** The validator monitor: name, type, timers and strings.




**Figure 11:** The alias service port, further down the same form.

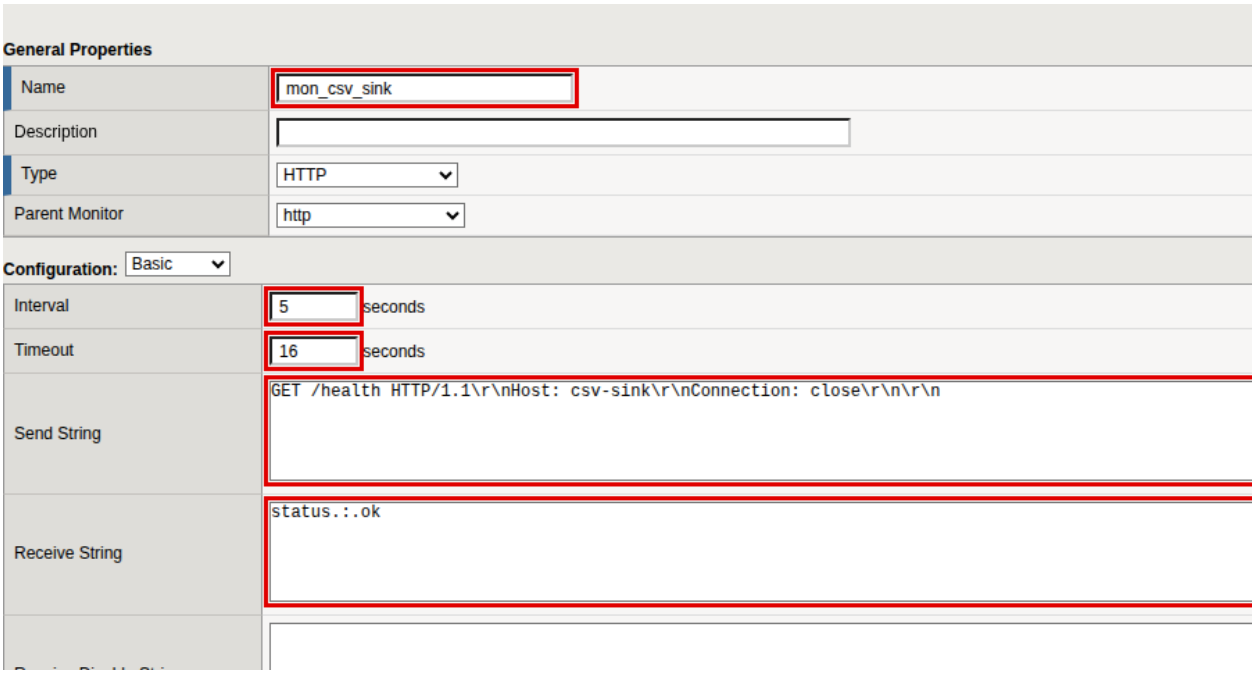



**Figure 12:** The upload application monitor.

## Pools

**Local Traffic › Pools › Pool List › Create**

| Field | pool_csv_validator_icap | pool_csv_sink |
|---|---|---|
| Health Monitors | mon_csv_validator | mon_csv_sink |
| New Members: Address | <lab-host> | <lab-host> |
| New Members: Service Port | 1345 | 8081 |

Click **Add** for the member, then **Finished**.

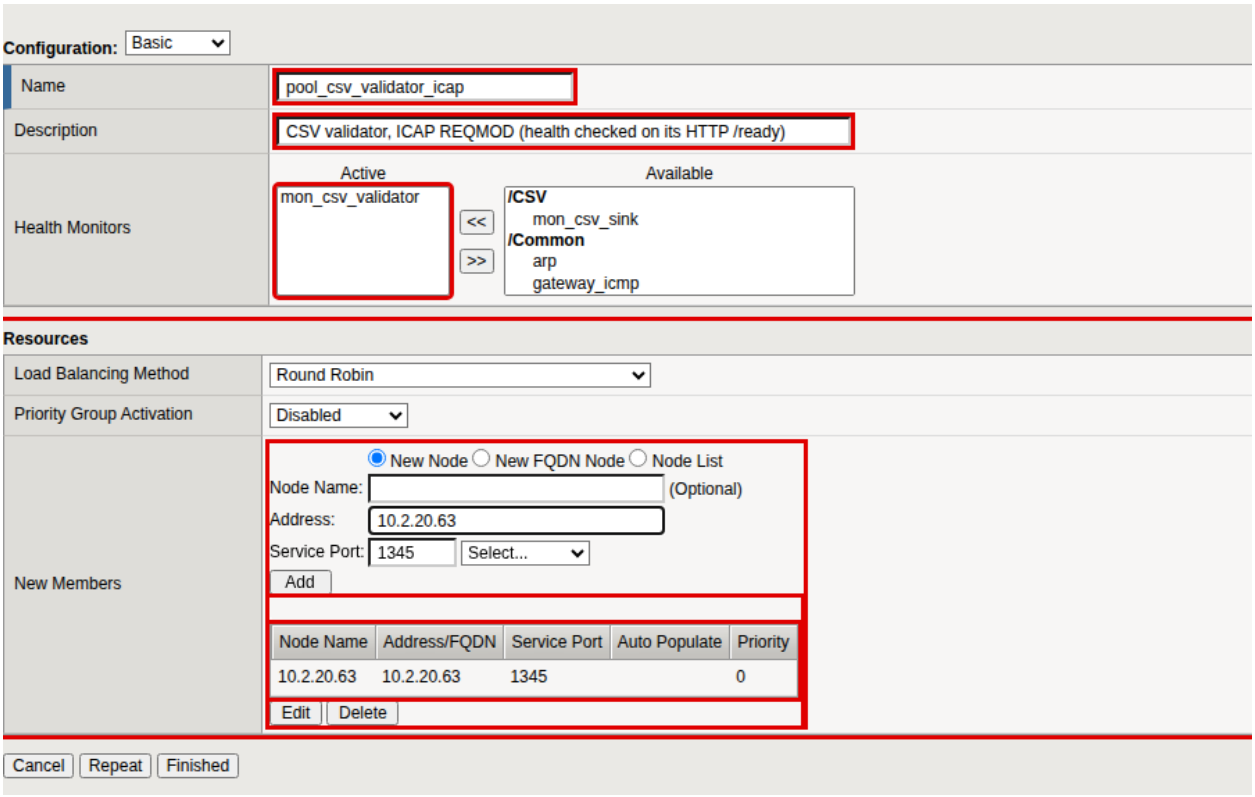



**Figure 13:** The validator pool.

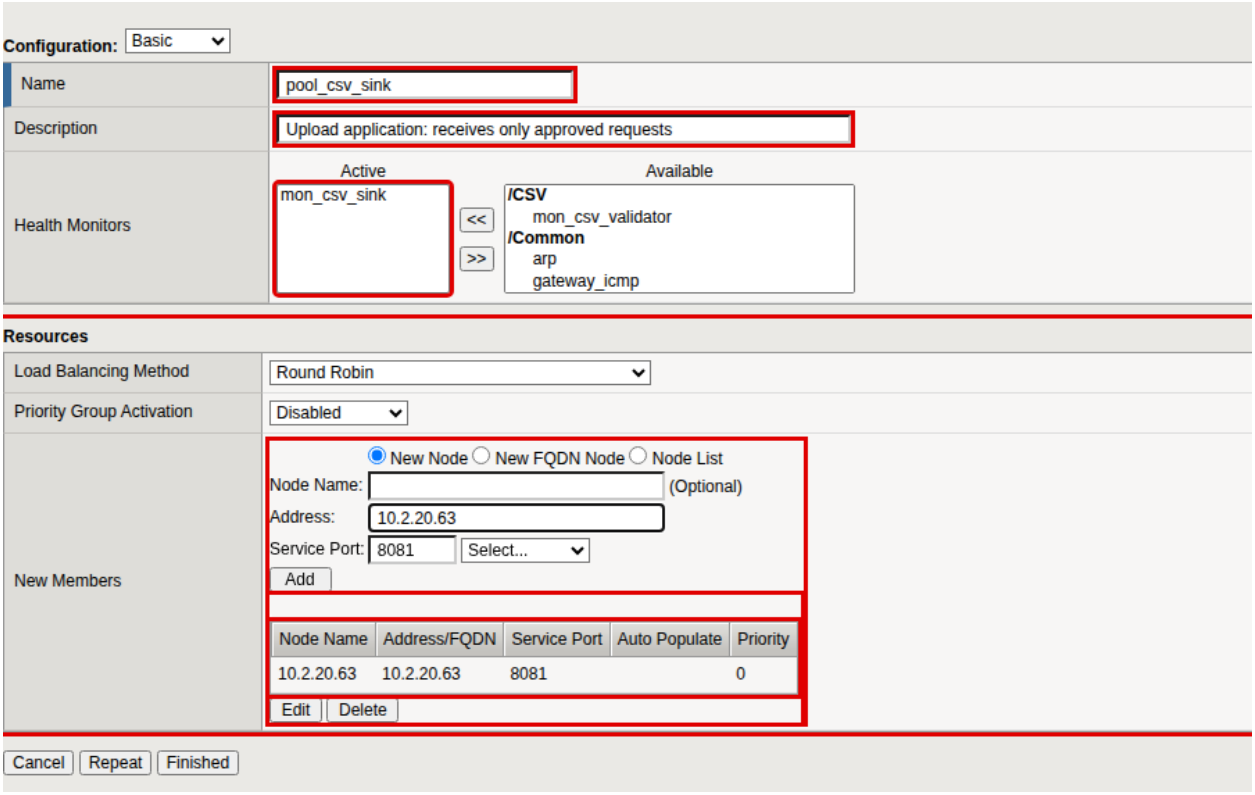



**Figure 14:** The upload application pool.

**You should see** both pools turn green within a few seconds. A red pool means the BIG-IP cannot reach the service from a self IP: check the route above, any firewall between them, and that the services are running.

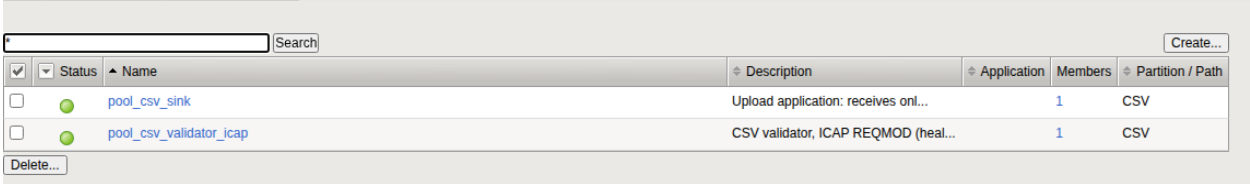



**Figure 15:** Both pools available.

# The Advanced WAF policy

This is the longest chapter. The policy allows exactly one thing, a `POST` to `/upload` carrying one file in a form field called `file`, and blocks everything else. Build it in the order given.

## Create the policy

**Security › Application Security › Security Policies › Policies List › Create**

| Field | Value |
|---|---|
| Policy Name | `waf_csv_lab` |
| Policy Type | *Security* |
| Policy Template | *Fundamental* |
| Virtual Server | *None*. Chapter 9 turns the policy on |
| Application Language | *Unicode (utf-8)* |
| Enforcement Mode | **Blocking** |
| Policy Building Learning Mode | **Disabled** |
| Signature Staging | **Disabled** |
| Differentiate between HTTP/WS and HTTPS/WSS URLs | **Disabled** |

Click **Save**.

Learning is disabled because nothing in this lab is learned from traffic: you set every control by hand. Left on its default, the policy would be free to relax itself.

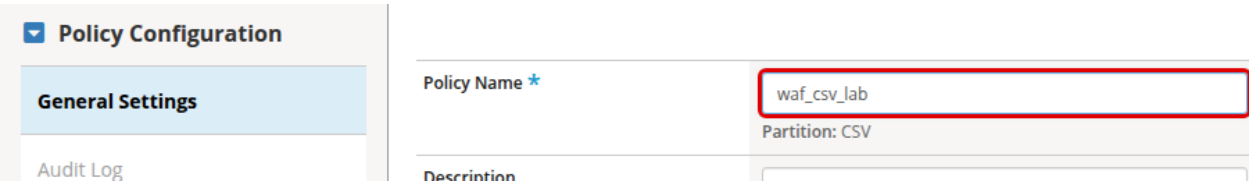



**Figure 16:** The policy create form: name, template and application language.

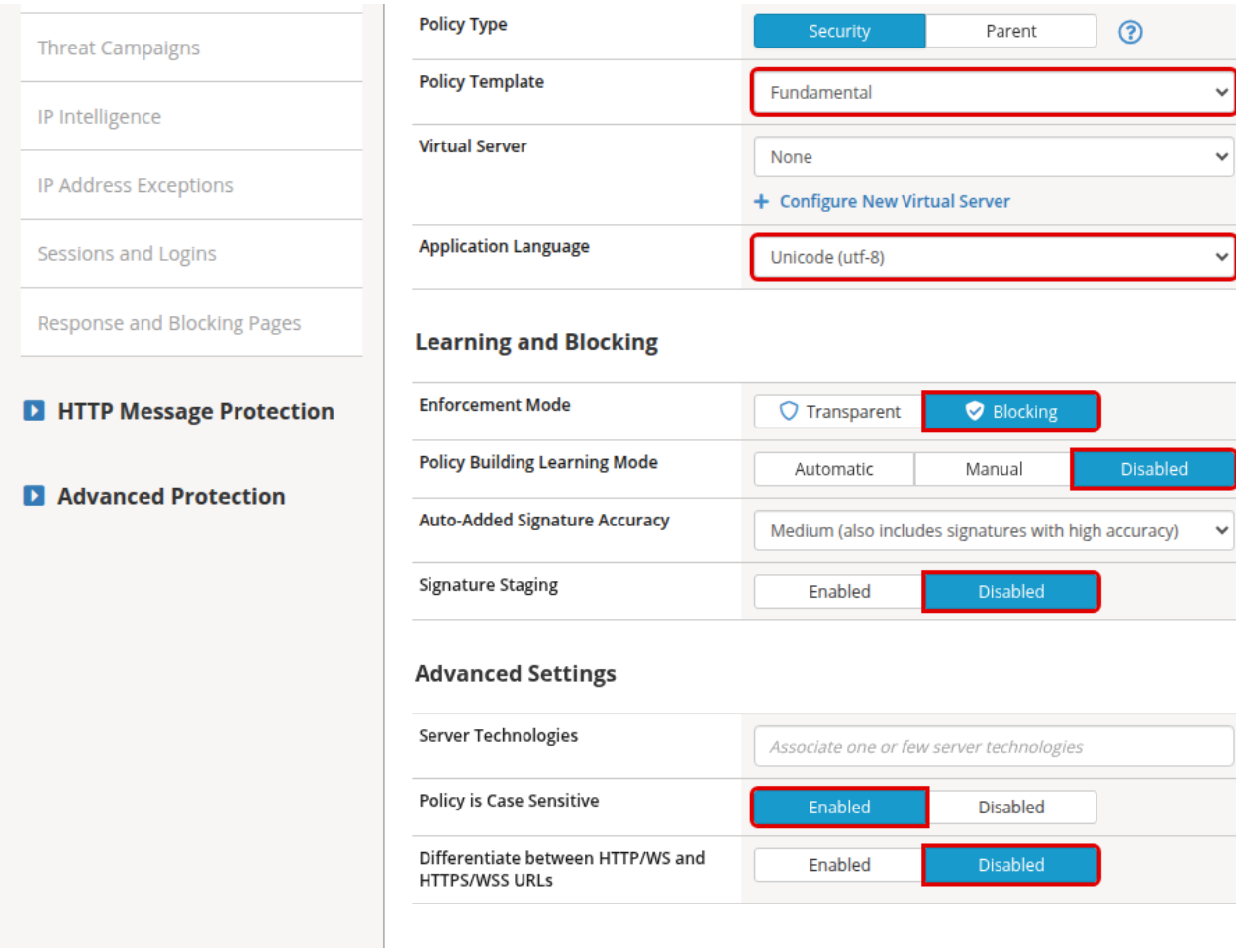



**Figure 17:** Learning and blocking: Blocking, learning disabled, signature staging disabled, URL differentiation disabled.

## General settings

Click the policy name. **General Settings** is the first page.

| Field | Value |
|---|---|
| Enforcement Readiness Period | `0` days |
| Allowed Response Status Codes | Add `411`, `413`, `415`, `422`, `502` and `504` to the defaults |
| Trust XFF Header | *Disabled* |

Click **Save**.

> **Do not skip the response codes.** The validator answers with those codes when it rejects a file. A code that is not on this list is a violation in its own right, and the WAF would replace the validator's answer with its own block page.

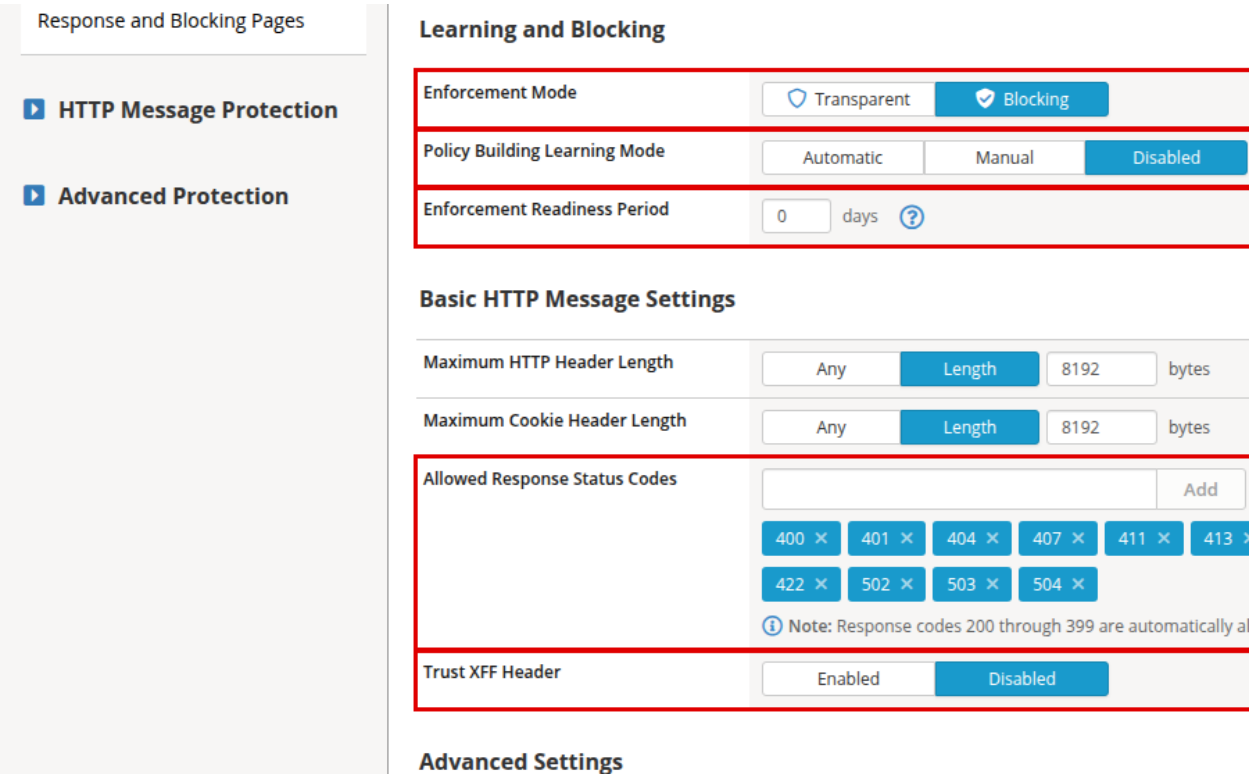



**Figure 18:** General Settings of the finished policy.

## Allowed URL

**Security › Application Security › URLs › Allowed URLs**

1. Tick the wildcard entry `*` and click **Delete**. Any other URL is now illegal.
2. Click **Create** and set the **Allowed URL Properties** selector to *Advanced*.

| Field | Value |
|---|---|
| URL | *Explicit*, `/upload` |
| Method | *POST* |
| Perform Staging | clear |
| Disallow File Upload of Executables | tick |
| Body is Mandatory | tick |

3. Click **Create**, open `/upload` again, and select the **Header-Based Content Profiles** tab. Make it read:

| Order | Request Header Name | Request Header Value | Request Body Handling |
|---|---|---|---|
| 1 | `Content-Type` | `multipart/form-data*` | *Form Data* |
| default | Any | Any | *Disallow* |

Delete the rows for `*json*` and `*xml*`. Click **Update**. A request with any other content type is now refused.

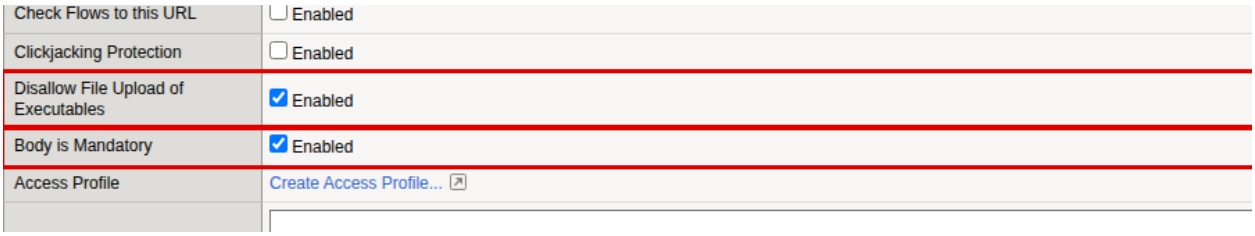



**Figure 19:** The upload URL: executables disallowed, body mandatory.

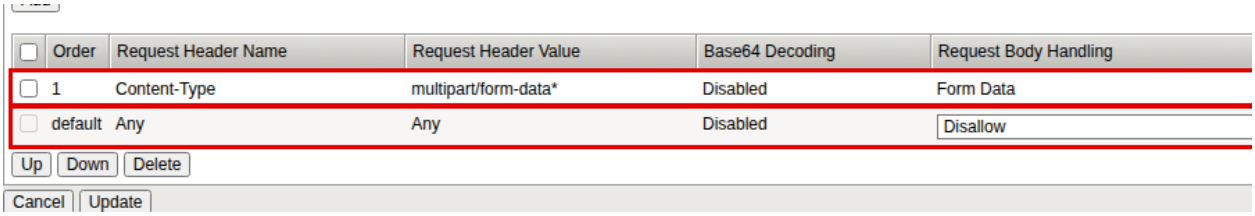



**Figure 20:** Content profiles: multipart/form-data allowed as Form Data, everything else disallowed.

## Allowed file type

The WAF's "file type" is the extension of the URL, not of the uploaded file. `/upload` has none, which the WAF calls `no_ext`. This entry carries the size limits for the whole request.

In the policy, expand **HTTP Message Protection** and click **File Types**.

1. Delete the wildcard `*`.
2. **Create › Allowed File Type**.

| Field | Value |
|---|---|
| File Type | *Explicit*, `no_ext` |
| Perform Staging | clear |
| URL Length | `64` |
| Request Length | `1310720` |
| Query String Length | `0` |
| POST Data Length | `1310720` |

`1310720` bytes is 1.25 MiB: a 1 MiB file plus the multipart wrapping.

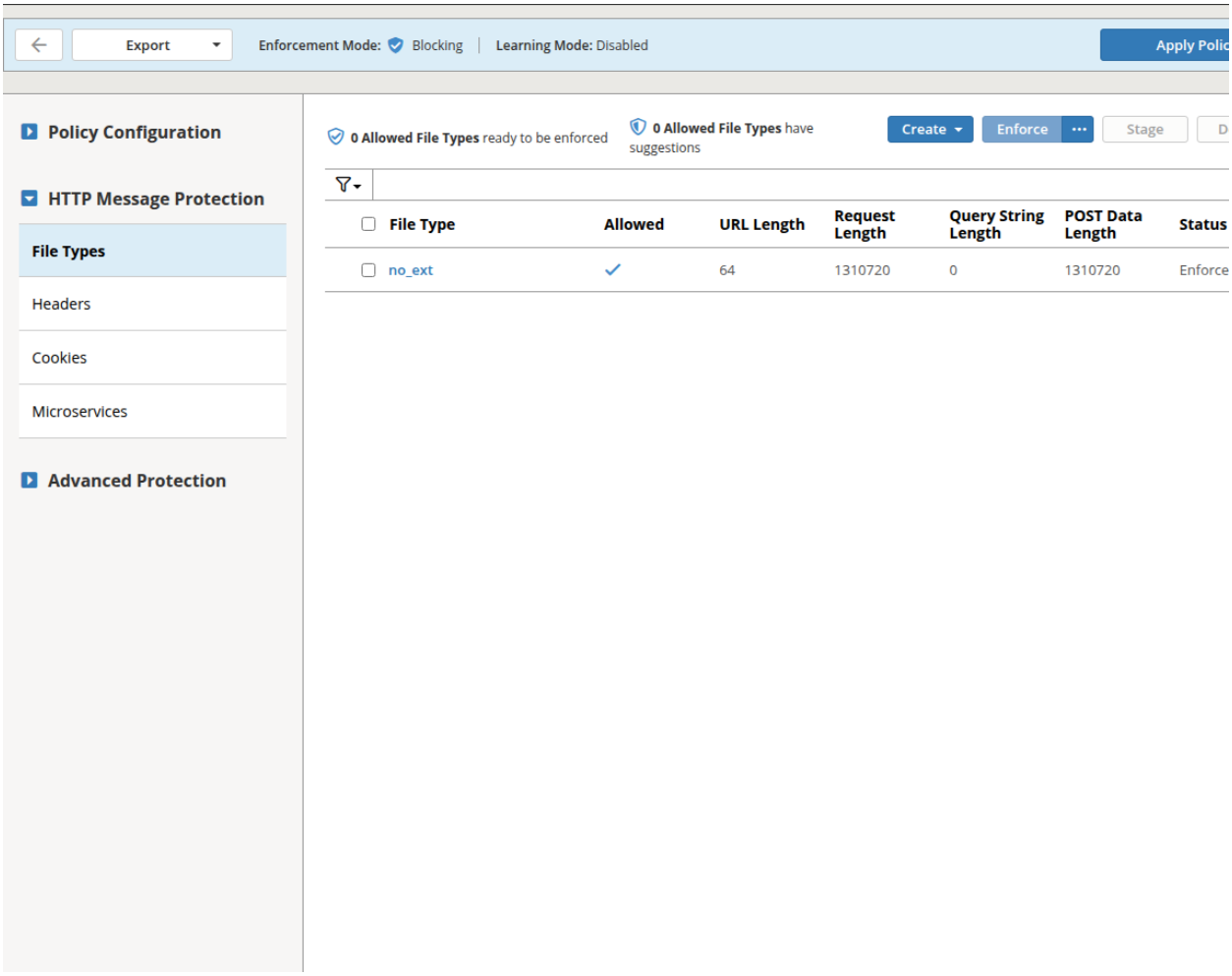



**Figure 21:** The single allowed file type and its limits.

## Parameters

**Security › Application Security › Parameters › Parameters List**

Delete the wildcard parameter `*`. From now on, any form field you have not declared is illegal, which is how a second file or a wrongly named field is blocked.

Create two parameters, both at **Parameter Level** *URL*, `POST /upload`.

### The file

| Field | Value |
|---|---|
| Parameter Name | *Explicit*, `file` |
| Location | *Form Data* |
| Perform Staging | clear |
| Is Mandatory Parameter | tick |
| Allow Empty Value | clear |
| Allow Repeated Occurrences | clear |

| Field | Value |
|---|---|
| Parameter Value Type | *User-input value* |
| Data Type | **File Upload** |
| Maximum Length | *Value*, `1048576` |
| Disallow File Upload of Executables | tick |
| Base64 Decoding | *Disabled* |
| Attack Signatures tab | clear *Check attack signatures* |

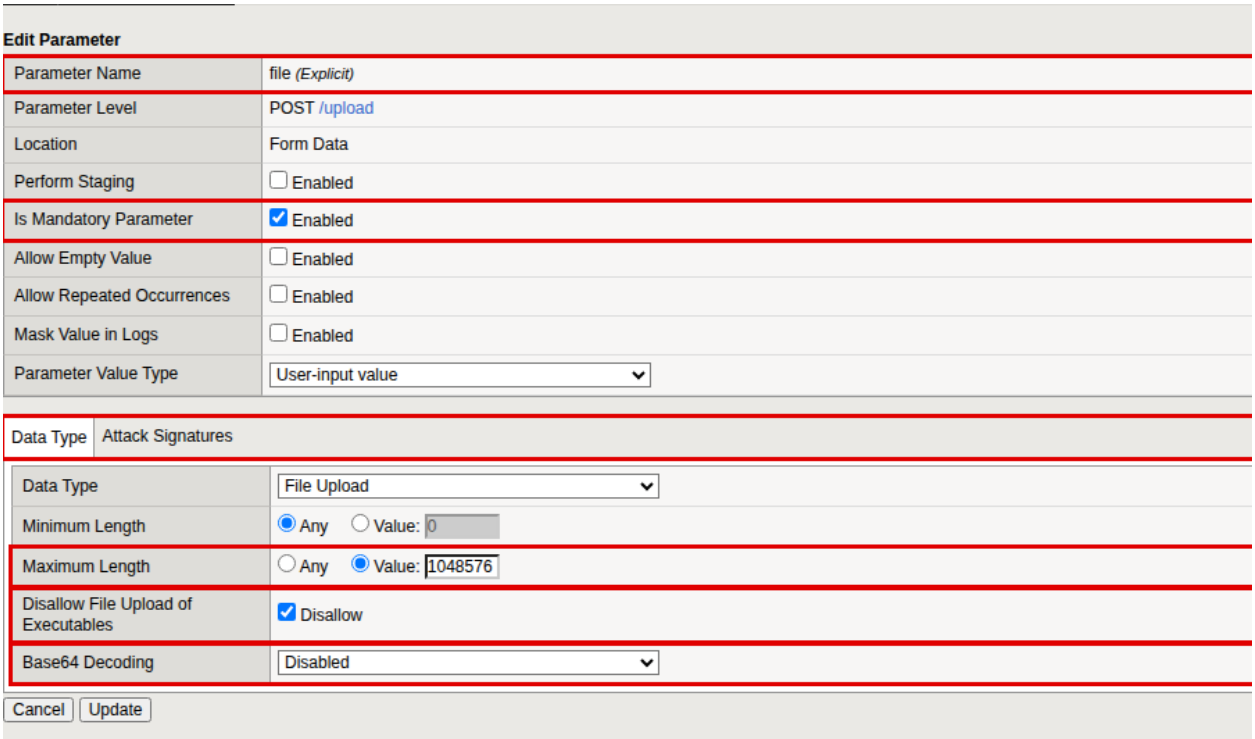



**Figure 22:** The file parameter.

### The file name

Advanced WAF presents the `filename` of an uploaded file as a form parameter of that name. Declaring it lets a regular expression enforce the extension.

| Field | Value |
|---|---|
| Parameter Name | *Explicit*, `filename` |
| Location | *Form Data* |
| Perform Staging | clear |
| Is Mandatory Parameter | tick |
| Allow Empty Value | clear |
| Parameter Value Type | *User-input value* |
| Data Type | *Alpha-Numeric* |
| Maximum Length | *Value*, `104` |

| Field | Value |
|---|---|
| Regular Expression | tick **Enable**, `^[A-Za-z0-9_-]{1,100}\.csv$` |
| Base64 Decoding | *Disabled* |

The pattern allows letters, digits, underscore and hyphen, then exactly one lower-case `.csv`. That refuses `report.txt`, `report.csv.exe` and any name containing a path.

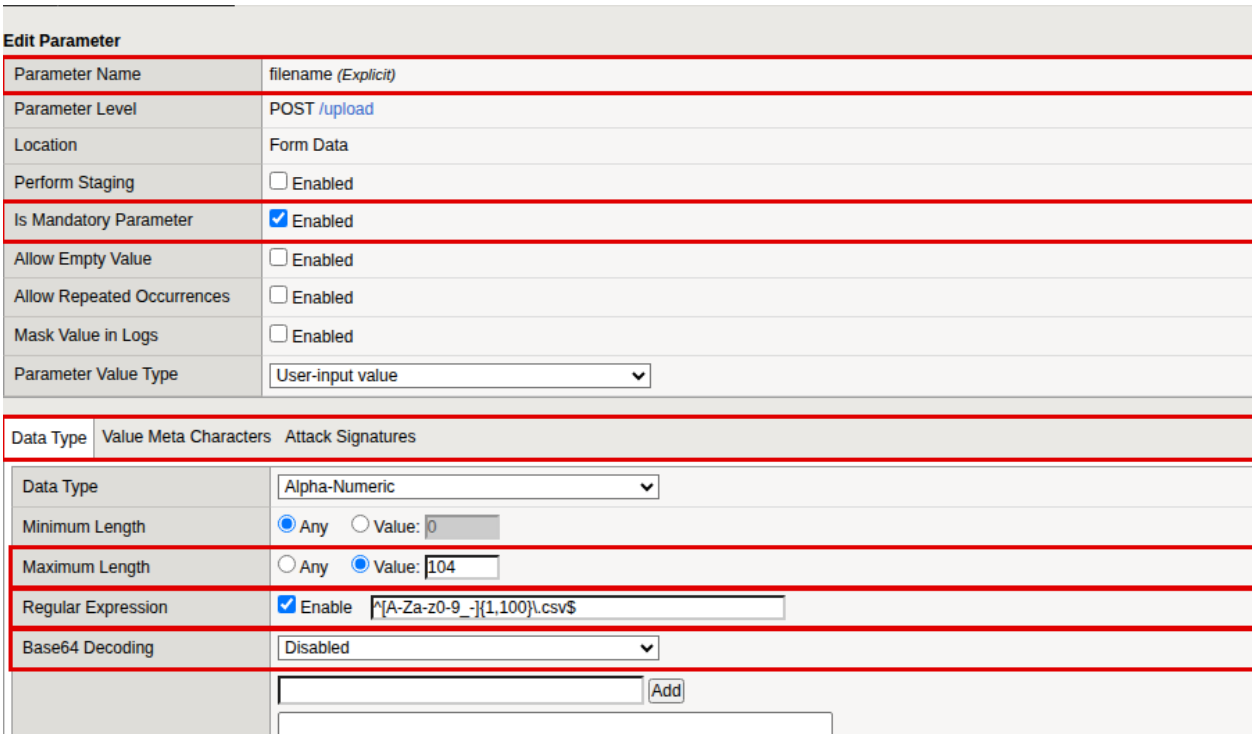



**Figure 23:** The filename parameter with its regular expression enabled.

**You should see** exactly two parameters and no wildcard.

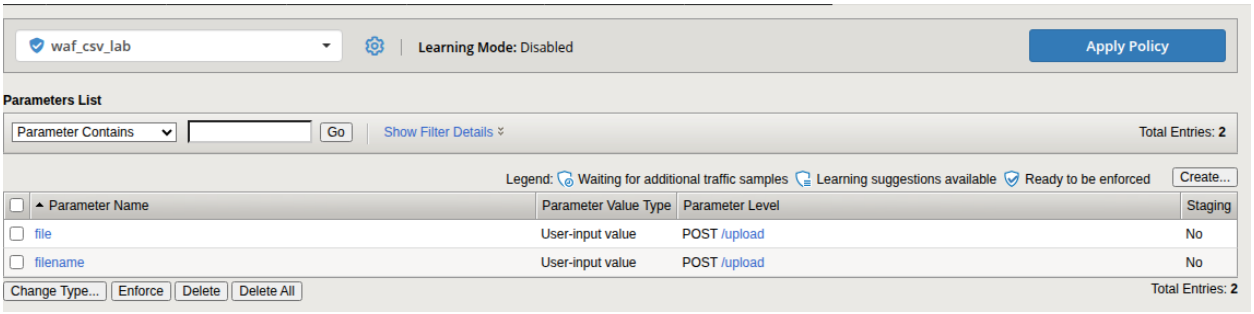



**Figure 24:** The finished parameter list.

## Learning and blocking settings

**Security › Application Security › Policy Building › Learning and Blocking Settings**

The page is a list of sections; click a name to open it. The bar at the top must read *Learning Mode: Disabled*. For every violation below, tick **Alarm** and **Block** and clear **Learn**.

| Section | Violations |
|---|---|
| Attack Signatures | Keep only *Generic Detection Signatures (High/Medium Accuracy)*, Alarm and Block. Clear **Enable Signature Staging** |
| Evasion technique detected | The violation, and these sub-violations: *Directory traversals*, *Multiple decoding* (2 passes), *Bad unescape*, *%u decoding* |
| File Types | *Illegal file type*, *Illegal URL length*, *Illegal request length*, *Illegal POST data length*, *Illegal query string length* |
| General Settings | *Illegal HTTP status in response*, *Request length exceeds defined buffer size*, *Failed to convert character* |
| Headers | *Illegal header length*, *Illegal method* |
| Cookies | *Illegal cookie length* |
| HTTP protocol compliance failed | The violation, and every sub-violation except *Host header contains IP address*, *High ASCII characters in headers* and *Bad host header value*. Set maximum headers `30`, maximum parameters `10`, maximum cookies `10` |
| Parameters | *Disallowed file upload content detected*, *Illegal parameter*, *Illegal parameter location*, *Illegal parameter value length*, *Illegal empty parameter value*, *Illegal repeated parameter name*, *Null in multi-part parameter value*, *Illegal parameter data type*, *Illegal parameter numeric value*, *Illegal static parameter value*, *Mandatory parameter is missing*, **Parameter value does not comply with regular expression** |
| URLs | *Illegal URL*, *Disallowed file upload content detected in body*, *Illegal request content type*, *Mandatory request body is missing* |

Click **Save**.

> **The one most often missed.** *Parameter value does not comply with regular expression* is off by default. Without it the file name pattern is checked and then ignored, and `report.txt` passes the WAF.

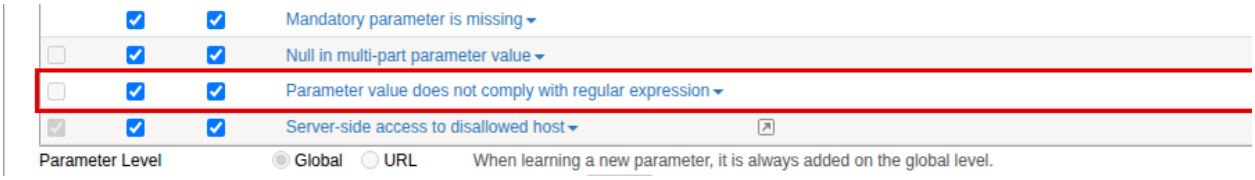



**Figure 25:** Learning and Blocking Settings. Learning Mode reads Disabled.

## Blocking response page

By default a blocked request gets an HTML page with status 200. The lab returns JSON with status 403 instead, which is easier for a client program, and for the tests, to recognise.

In the policy, **Response and Blocking Pages › Blocking Page Default**.

| Field | Value |
|---|---|
| Response Type | *Custom Response* |
| Response Headers | `HTTP/1.1 403 Forbidden`, `Content-Type: application/json`, `Connection: close`, one per line |
| Response Body | `{"blocked_by":"bigip-asm","support_id":"<%TS.request.ID()%>"}` |

Click **Save**.




**Figure 26:** The custom blocking response.

## Apply the policy

Click **Apply Policy**, the blue button at the top right, and confirm.

**You should see** `waf_csv_lab` in the Policies List in *Blocking* mode with no pending-changes marker.

# Antivirus scanning

Advanced WAF can hand each uploaded file to an antivirus scanner over ICAP and hold the request until the scanner answers. Three settings are involved: one for the whole BIG-IP and two in the policy.

## Point the BIG-IP at the scanner

**Security › Options › Application Security › Integrated Services › Anti-Virus Protection**

| Field | Value |
|---|---|
| Server Host Name/IP Address | `<av-host>` |
| Server Port Number | `1344` |
| Guarantee Enforcement | tick **Enabled** |

Click **Save**.

*Guarantee Enforcement* makes the WAF block an upload when the scan cannot be performed. Without it, an unreachable scanner means unscanned files are delivered.

This is one setting for the whole BIG-IP, shared by every WAF policy on it. It is not part of the `CSV` partition.




**Figure 27:** The device-wide antivirus server.

## Restart the Application Security service

> **This step is required and the Configuration utility does not prompt for it.** On 21.1.0.2 the WAF reads the antivirus server address only when the service starts. In testing, after the address was first set, the BIG-IP made no connection to the scanner, logged nothing, and delivered a file containing the test virus.

Log in to the BIG-IP command line and run:

```
bigstart restart asm
```

Application Security is unavailable for about a minute. Repeat this whenever you change the antivirus server address or port.

## Turn scanning on in the policy

1. Open `waf_csv_lab` › **General Settings**. Under **Integrated Services**, **Anti-Virus Protection**, tick **Scan HTTP Uploads**. Click **Save**.
2. **Policy Building › Learning and Blocking Settings › Antivirus.** Set *Virus detected* to **Alarm** and **Block**, with **Learn** clear. Click **Save**.
3. Click **Apply Policy**.

You prove the scan in chapter 9, once there is a virtual server to send uploads to.

# The validator hop

The BIG-IP feature that sends a request to an outside service for a verdict is called *request adapt*. It speaks ICAP, the same protocol the antivirus uses. Four objects make it work, and they must be created in this order, because each one names the one before.

```
inner virtual server
    -> request-adapt profile
        -> internal virtual server
            -> ICAP profile + pool of validators
```

## ICAP profile

**Local Traffic › Profiles › Services › ICAP › Create**

| Field | Value |
|---|---|
| Name | `icap_csv_validator` |
| Parent Profile | `icap` |
| URI | Tick **Custom**, `icap://${SERVER_IP}:${SERVER_PORT}/validate` |
| Preview Length | Tick **Custom**, `0` |

Type the URI exactly as shown, including `${SERVER_IP}` and `${SERVER_PORT}`. The BIG-IP replaces them with the pool member it picks.




**Figure 28:** The ICAP profile.

## Internal virtual server

An internal virtual server has no address. It exists only so that a request-adapt profile has somewhere to send traffic.

**Local Traffic › Virtual Servers › Virtual Server List › Create**

Set the **Configuration** selector to *Advanced* so that the ICAP field is shown.

| Field | Value |
|---|---|
| Name | `ivs_csv_validator` |
| Type | **Internal** |
| ICAP Profile | `icap_csv_validator` |

| Field | Value |
|---|---|
| Source Address Translation | *Auto Map* |
| Default Pool | `pool_csv_validator_icap` |

Click **Finished**. The type cannot be changed afterwards.

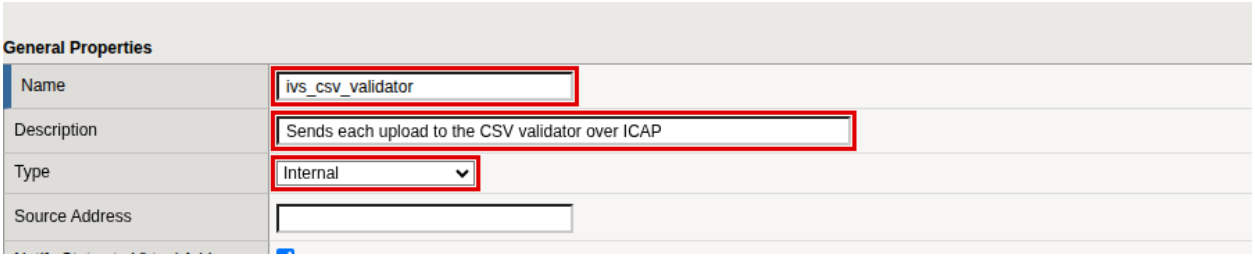



**Figure 29:** Name and type. The address fields disappear when Internal is chosen.




**Figure 30:** The ICAP profile and source address translation.

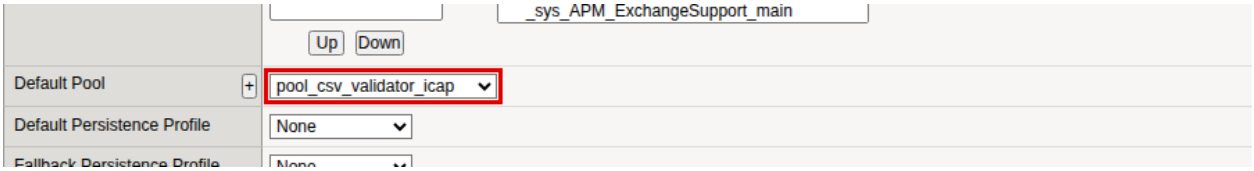



**Figure 31:** The validator pool, at the foot of the form.

## Request-adapt profile

**Local Traffic › Profiles › Services › Request Adapt › Create**

| Field | Value |
|---|---|
| Name | `reqadapt_csv_lab` |
| Parent Profile | `requestadapt` |
| Enabled | Tick **Custom**, keep ticked |
| Internal Virtual Name | Tick **Custom**, `/CSV/ivs_csv_validator` |
| Preview Size | Tick **Custom**, `0` |
| Timeout (ms) | Tick **Custom**, `20000` |

| Field | Value |
|---|---|
| Service Down Action | Tick **Custom**, **Reset** |

**The default fails open.** *Service Down Action* is *Ignore* in the parent profile: if the validator is down, requests go straight to the application without a verdict. *Reset* closes the connection instead.

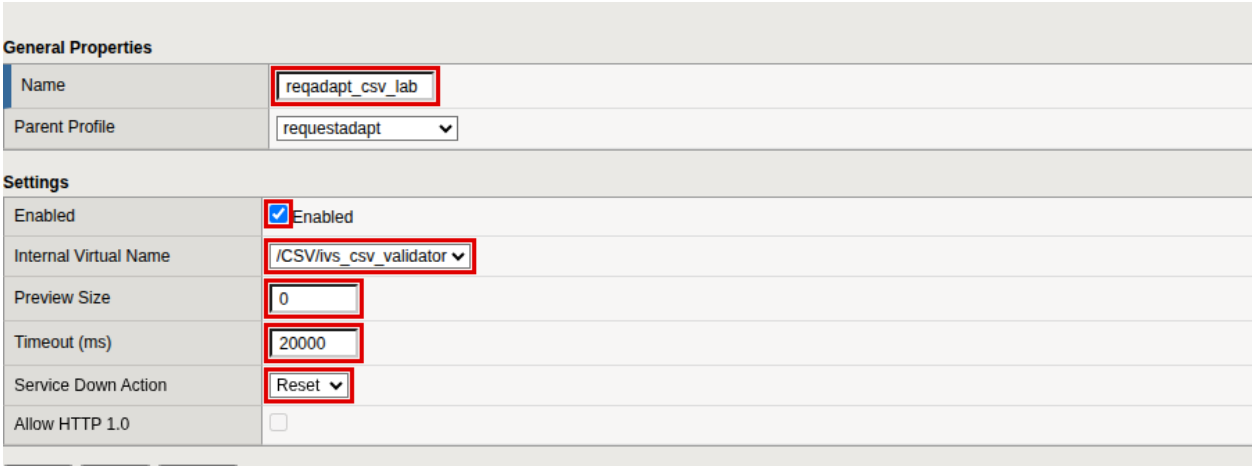



**Figure 32:** The request-adapt profile. Service Down Action is Reset.

## Inner virtual server

This virtual server runs request adapt and then load balances to the upload application. It is enabled on no VLAN, so nothing on the network can reach it; only the outer virtual server you build next can.

**Local Traffic › Virtual Servers › Virtual Server List › Create**, with the **Configuration** selector on *Advanced*.

| Field | Value |
|---|---|
| Name | `vs_csv_inner` |
| Type | *Standard* |
| Destination Address/Mask | `<inner-ip>` |
| Service Port | `80` |
| HTTP Profile (Client) | `http` |
| Request Adapt Profile | `reqadapt_csv_lab` |
| VLAN and Tunnel Traffic | **Enabled on…**, and leave the *Selected* list **empty** |
| Source Address Translation | *Auto Map* |
| Default Pool | `pool_csv_sink` |

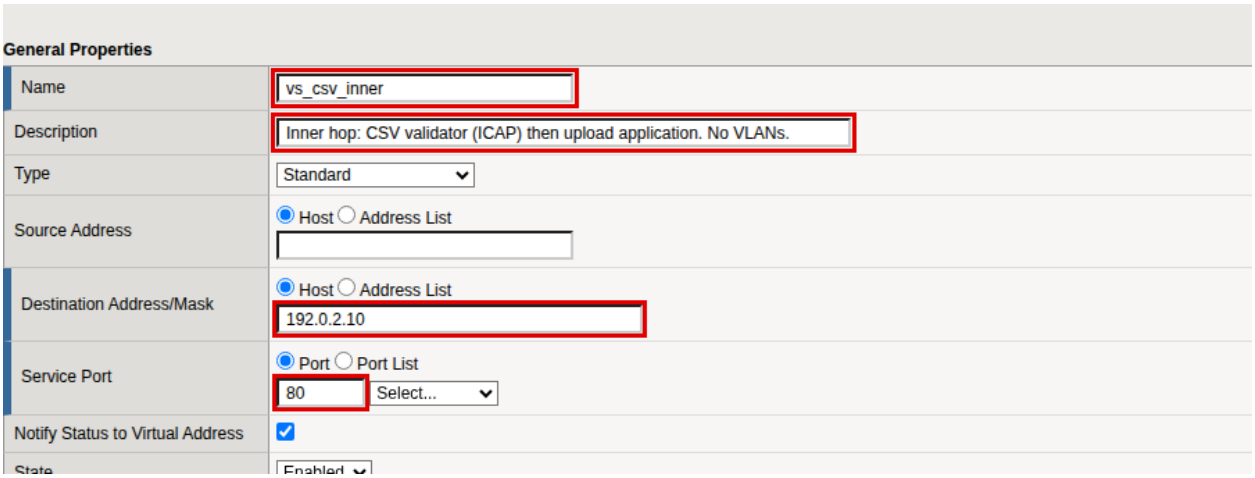



**Figure 33:** Name, address and port.




**Figure 34:** The HTTP profile.

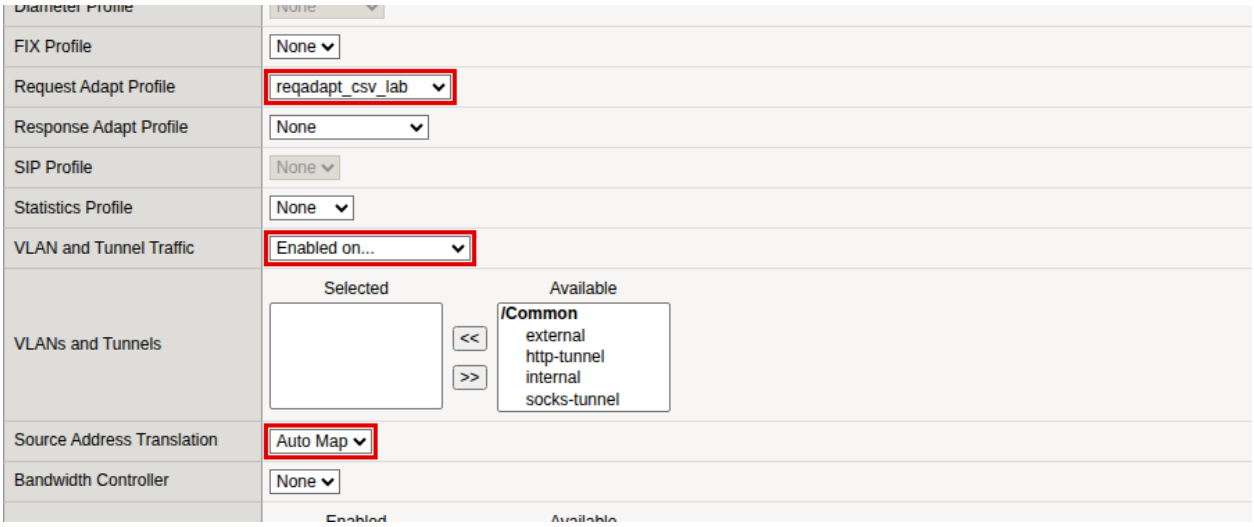



**Figure 35:** Request adapt, no VLANs, Auto Map and the application pool.

**You should see** `ivs_csv_validator` and `vs_csv_inner` in the Virtual Server List, both available.

## Why there are two virtual servers

It would be simpler to put request adapt on the same virtual server as the WAF policy. On 21.1.0.2 that runs request adapt *first*: the validator sees every request before the WAF has had the chance to block it. Splitting the work across two virtual servers fixes the order. The outer one runs the WAF and the antivirus; only what survives is forwarded to the inner one.

# The entry point

Two objects remain: a policy that strings the steps together, and the virtual server your clients connect to.

## LTM policy

One rule with two actions: turn on the WAF policy for the request, then forward the request to the inner virtual server.

**Local Traffic › Policies › Policy List › Create**

1. **Policy Name** `ltm_csv_lab`. **Strategy** *Execute first matching rule*. Click **Create Policy**.
2. Under **Rules**, click **Create**. **Name** `waf_then_inner`. Leave the condition as *All traffic*.
3. Under **Do the following when the traffic is matched**, add:
   - **Enable** › *asm* › for policy `/CSV/waf_csv_lab` › at *request* time
   - **Forward traffic** › to *virtual server* › `/CSV/vs_csv_inner` › at *request* time
4. Click **Save**, then **Save Draft**, then **Publish**.

If you prefer the command line, this produces the same policy and was tested on 21.1.0.2:

```
tmsh -c 'cd /CSV; create ltm policy Drafts/ltm_csv_lab strategy first-match
  controls add { asm forwarding } requires add { http }
  rules add { waf_then_inner { ordinal 1 actions add {
    0 { asm enable policy /CSV/waf_csv_lab }
    1 { forward select virtual /CSV/vs_csv_inner } } } }'
tmsh -c 'cd /CSV; publish ltm policy Drafts/ltm_csv_lab'
```

**You should see** the published policy read as below.

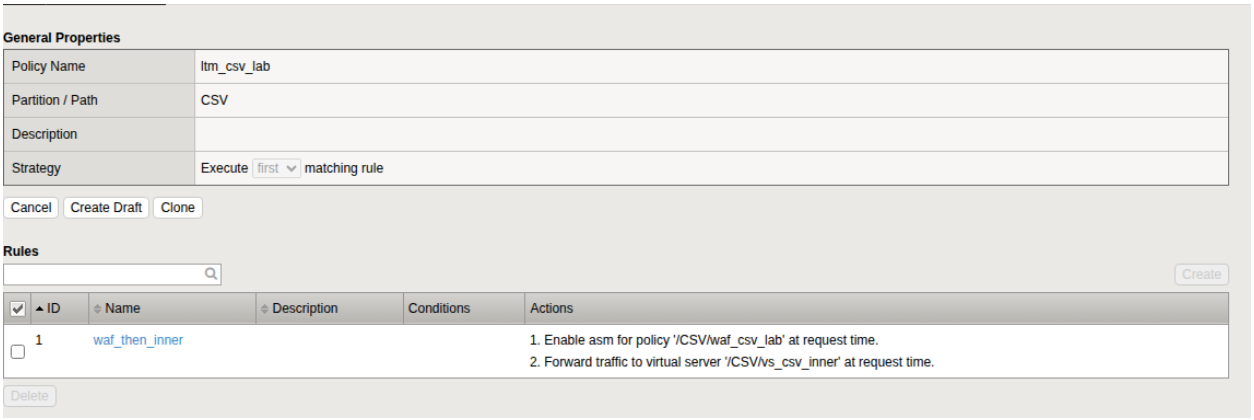



**Figure 36:** The published policy: one rule, two actions.

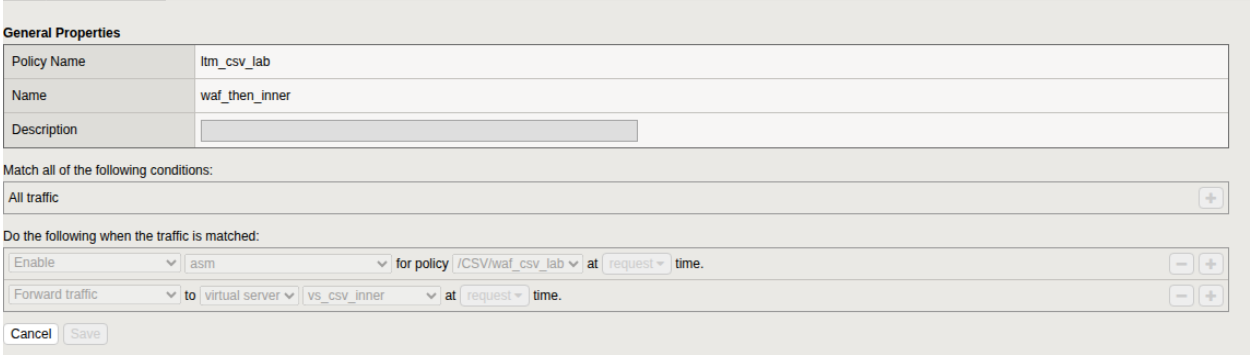



**Figure 37:** The rule in the policy editor.

## Outer virtual server

**Local Traffic › Virtual Servers › Virtual Server List › Create**

| Field | Value |
|---|---|
| Name | `vs_csv_lab` |
| Type | *Standard* |
| Destination Address/Mask | `<vip>` |
| Service Port | `443` |
| HTTP Profile (Client) | `http_csv_lab` |
| SSL Profile (Client) | move `clientssl_csv_lab` to *Selected* |
| Source Address Translation | *Auto Map* |
| Default Pool | *None* |

Click **Finished**, then open the virtual server again:

1. **Resources** tab › **Policies** › **Manage**. Move `ltm_csv_lab` to *Enabled*. Click **Finished**.
2. **Security › Policies** tab. Set **Log Profile** to *Enabled* and move *Log illegal requests* to *Selected*. Click **Update**.
3. Return to **Security › Application Security › Security Policies** and click **Apply Policy** for `waf_csv_lab` if it shows pending changes.

There is no default pool because the LTM policy forwards every request. For the same reason this virtual server shows status *unknown* (blue), not green. The health of the lab is on the two pools.

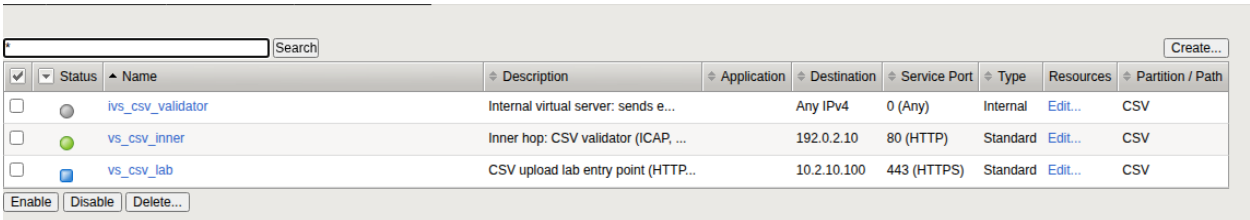



**Figure 38:** The three virtual servers.

## The first uploads

From the test host:

```
cd test
u() { curl -s --cacert tls/ca.pem -w " [%{http_code}]\n" \
        -F "file=@files/$1;type=text/csv" https://<vip>/upload; }

u ok_basic.csv           # delivered
u bad_ext.txt            # blocked by the WAF
u av_eicar_in_csv.csv    # blocked by the antivirus
u bad_formula_eq.csv     # rejected by the validator
```

**You should see**, in order:

```
{"accepted":true,"upload_id":"lab-...","status":"approved","sha256":"...","bytes":37} [200]
{"blocked_by":"bigip-asm","support_id":"..."} [403]
{"blocked_by":"bigip-asm","support_id":"..."} [403]
{"accepted":false,"upload_id":"lab-...","status":"rejected","reason_code":"CSV_FORMULA_CELL",...}
  [422]
```

| If you see | It means |
|---|---|
| `[400]` with `FILENAME_EXTENSION` for `bad_ext.txt` | The validator answered before the WAF. Request adapt is on the outer virtual server, or the file name regular expression or its violation is not enabled (chapter 6) |
| `[200]` for `av_eicar_in_csv.csv` | The file was not scanned. Restart the Application Security service (chapter 7) |
| `[403]` for `ok_basic.csv` | The scanner cannot be reached, or a response code is missing from the policy (chapter 6). Look up the support ID as described in chapter 11 |
| A connection reset | The validator pool is down |
| `curl: (60)` | The `--cacert` option is missing. That is certificate validation working |

# Acceptance tests

The bundle includes a test runner that sends 43 uploads through the BIG-IP, checks each response, and then asks the upload application what it actually received.

## Run the tests

On the test host:

```
BIGIP_URL=https://<vip>/upload SINK_URL=http://127.0.0.1:8081 \
  MODE=bigip test/run-tests.sh
```

The client validates the BIG-IP's certificate against `test/tls/ca.pem`. Add `OPS=1` to include the outage tests in the next section. They stop and start containers with `podman`, so run them on VM 1 as `csvlab`, and set `ICAP_AV_CONTAINER=` (empty) because the antivirus is on VM 2; do that one outage test by hand as described below.

**You should see** a line per case and a final result:

```
accept_basic               bigip  200 PASS approved          lab-6d963ac74750d7a2
bigip_ext_txt              bigip  403 PASS BIGIP             support:7553261307881850587
av_eicar_in_valid_csv      bigip  403 PASS BIGIP             support:7553261307881850579
val_formula_eq             bigip  422 PASS CSV_FORMULA_CELL  lab-1ceb08078219e99f
...
  10 accepted files present, 28 rejected files absent
RESULT: 84 passed, 0 failed
```

With `OPS=1` the total is 92. The full record of each run is written to `test/results/`.

## What is tested

| Group | Cases | Expected |
|---|---|---|
| Accepted | 10: plain, quoted, embedded comma, embedded line break, semicolon, tab, accented UTF-8, signed numbers, phone-like values, CRLF line endings | 200 `approved`; the file is in the application |
| Stopped by the WAF | 13: `.txt`, `.csv.exe`, Windows and Linux executables renamed `.csv`, oversized request, oversized file, wrong field name, no file, two files, wrong content type, malformed body, UTF-16, empty file | 403 with a support ID |
| Stopped by the antivirus | 2: the EICAR test file, and the EICAR string inside an otherwise valid CSV | 403 with a support ID |
| Rejected by the validator | 18: not really CSV (JSON, HTML), invalid UTF-8, broken quoting, uneven columns, blank lines, unsupported delimiter, six kinds of formula cell, markup, a `javascript:` link | 4xx with an upload ID and a reason code |

| Group | Cases | Expected |
|---|---|---|
| Delivery check | all of the above | Every accepted file's hash is in the application. No other file's hash is |
| Certificate validation | 3 | The lab certificate authority is accepted. The system trust store and an unrelated authority are refused |

The second antivirus case is the important one. That file is a perfectly valid CSV, so the validator approves it. Only the antivirus can stop it, and it proves the scan really happens. It is detected by the lab's own `Lab-EICAR-Anywhere` signature (chapter 4); the official EICAR rule would not match a test string embedded in another file.

## Outage tests

These prove the lab fails closed. Run them with `OPS=1`, or by hand: stop one container, upload `ok_basic.csv`, start the container again, and wait a few seconds for the BIG-IP monitors to recover.

| Stop this | You should see | Why |
|---|---|---|
| `csv-icap-av` | 403 with a support ID | *Guarantee Enforcement* blocks uploads that cannot be scanned |
| `csv-validator` | Connection reset | *Service Down Action* is *Reset* |
| `csv-sink` | No response | The pool has no available member |

In no case does the upload return 200 or appear in the application.

`OPS=1` also checks ten simultaneous uploads, that repeated uploads get different upload IDs, that a client disconnecting mid-upload leaves the lab healthy, and two attempts to cheat:

- **Posting straight to the application**, with and without forged approval headers. The application answers 403: the signature does not verify.
- **Sending forged approval headers through the BIG-IP.** The validator strips them and judges the file as usual.

## Acceptance criteria

The lab is working when all of these are true.

1. A valid CSV reaches the application.
2. An invalid CSV, a CSV containing a formula, and the EICAR test file do not.
3. An executable renamed `.csv` is blocked.
4. Extra or unexpected form fields are blocked.
5. With any one service stopped, no upload is delivered.
6. A request that did not pass the validator is refused by the application.
7. The test client validates the BIG-IP's certificate.
8. Every block has a BIG-IP support ID, every validator verdict has an upload ID, and the logs tie them to the file's hash.

# Operations

## Finding out what happened to an upload

| Outcome | The client sees | Where the detail is |
|---|---|---|
| Blocked by the WAF | 403 and a support ID | **Security › Event Logs › Application › Requests**. Filter by the support ID. The entry names the violation, for example *Parameter value does not comply with regular expression* on parameter `filename` |
| Blocked by the antivirus | 403 and a support ID | The same log, violation *Virus detected*. On the antivirus host, `podman logs csv-icap-av` shows `action=blocked virus=<name>` at the same second |
| Rejected by the validator | 4xx, an upload ID and a reason code | `podman logs csv-validator`: a line with `"event": "rejected"`, the client address and the file hash |
| Delivered | 200, an upload ID and the file hash | On VM 1: validator log, `"event": "validated"`; application log, `"event": "received"`; `curl http://127.0.0.1:8081/received?upload_id=<i` returns the record |

The request log in this lab records blocked requests only. To see allowed requests there as well, change the log profile on `vs_csv_lab` to *Log all requests*.

## Troubleshooting

| Symptom | Cause | Fix |
|---|---|---|
| Every upload, including good ones, gets 403 | The antivirus cannot be reached | Check the container is running and that TCP 1344 is open from the BIG-IP |
| The EICAR test file is delivered | The Application Security service was not restarted after the antivirus server was set | `bigstart restart asm`, then repeat the upload |
| `report.txt` is rejected by the validator, not the WAF | The file name pattern is not being enforced | Chapter 6: the **Enable** box beside the regular expression, and the *Parameter value does not comply with regular expression* violation |
| The validator's rejections arrive as 403 block pages | A response code is missing | Chapter 6, *Allowed Response Status Codes* |

| Symptom | Cause | Fix |
|---|---|---|
| Every upload is reset | The validator pool is down | Check `curl http://<lab-host>:8080/ready` and the pool status |
| The application answers 403 `BAD_SIGNATURE` | The validator and the application have different secrets | Recreate both containers from the same `.env` |
| A pool stays red although the service answers locally | The BIG-IP cannot reach the host from a self IP | Chapter 5, routes; check for a host firewall |
| The antivirus container stops in full mode | It could not download signatures, or ran out of memory | Check outbound HTTPS and give it 2 GB, or use test mode |
| The policy shows pending changes | Something was edited after the last apply | Click **Apply Policy** |

## Things to try

The lab is yours to change. Some experiments that teach the most:

- **Relax one control and re-run the tests.** Clear the **Enable** box on the file name pattern. `bigip_ext_txt` now fails the test, because the request gets past the WAF and it is the validator that refuses it. The file still does not arrive: that is defence in depth.
- **Move request adapt to the outer virtual server** and watch the validator receive requests the WAF should have stopped.
- **Change *Service Down Action* to *Ignore***, stop the validator, and see a file delivered with no verdict. Change it back.
- **Switch the WAF policy to *Transparent*.** Violations are logged but nothing is blocked, and the validator and antivirus still do their part.
- **Upload your own CSV files**, including exports from the application you have in mind. Appendix C lists the rules they are judged against.

## Removing the lab

With the **Partition** selector on `CSV`, delete in this order. Each delete fails while something still refers to the object.

1. Virtual server `vs_csv_lab`.
2. LTM policy `ltm_csv_lab`.
3. Virtual server `vs_csv_inner`.
4. Request Adapt profile `reqadapt_csv_lab`.
5. Virtual server `ivs_csv_validator`, then ICAP profile `icap_csv_validator`.
6. Security policy `waf_csv_lab`.
7. **Security › Options › Application Security › Integrated Services › Anti-Virus Protection**: clear the server address and **Save**.
8. Both pools, the node, both monitors.
9. Profiles `http_csv_lab` and `clientssl_csv_lab`, certificate `csv_lab`, and any routes.
10. Partition `CSV`.

On the lab hosts:

```
podman-compose down -v                    # validator and application
(cd icap-av && podman-compose down -v)    # antivirus
```

# Show the lab to others

Once the lab passes its tests you can use it to show, in about twenty minutes, what each layer contributes. The order below builds from the simplest control to the hardest to explain, and every step has one command and one thing to point at.

Have three windows open: a terminal on the test host, the BIG-IP request log (**Security › Event Logs › Application › Requests**), and a terminal on VM 2 running `podman logs -f csv-icap-av`.

## 1. A good file goes through

```
u ok_basic.csv
```

Point at: the `approved` response, and the record it creates in the upload application (on VM 1: `curl http://127.0.0.1:8081/received | tail -c 400`).

## 2. The WAF stops what is not an upload

```
u bad_ext.txt
u bad_pe_renamed.csv
```

Point at: the support ID in each 403, then the same ID in the request log. The first entry says the file name failed the regular expression; the second says the content was a Windows executable, whatever its name. The validator and the antivirus never saw either request.

## 3. The antivirus stops a valid CSV that carries a payload

```
u av_eicar_in_csv.csv
```

Point at: this file is a correct CSV, so the CSV rules approve it. The 403 comes from the WAF's *Virus detected* violation, and the scanner window shows the matching `action=blocked virus=Lab-EICAR-Anywhere` line at the same second. This is the step that proves the BIG-IP really sends files to the scanner. Say what the signature is: a lab rule matching the test string anywhere, because the official EICAR rule needs the string to be the whole file; real malware signatures do match inside files.

## 4. The validator judges what the WAF cannot

```
u bad_formula_eq.csv
u bad_missing_column.csv
```

Point at: these get a 422 with a reason code, not a 403. The WAF passed them, because they are well-formed uploads; the verdict came from the validator over ICAP, and the BIG-IP returned it to the client. The application was never contacted.

## 5. Nothing gets in behind the BIG-IP

On VM 1:

```
curl -s -F "file=@test/files/ok_basic.csv;type=text/csv" http://127.0.0.1:8081/upload
```

And from any other host, `curl -m 5 http://<lab-host>:8081/health`.

Point at: locally, the application answers 403 with `NO_APPROVAL`, because the request carries no validator signature. From another host, the connection does not open at all: the firewall admits only the BIG-IP.

## 6. Every service fails closed

On VM 2: `podman stop csv-icap-av`, then `u ok_basic.csv`.

Point at: a 403 with a support ID for a perfectly good file, because the scan could not be performed. Start the container again and repeat the upload; it is approved. Do the same with `csv-validator` on VM 1 and show the connection reset instead.

## 7. Where the evidence is

Pick one approved upload ID and show it in three places: the validator log, the application's record, and the file hash that matches in both. Then pick one support ID and show it in the request log. This is what an incident responder would follow.

## What to say it is not

Say plainly that the three services are sample code written for this lab, that antivirus test mode detects only the test string, and that the file rules in Appendix C are a policy, not a proof that a file is safe. Showing the limits is part of the demonstration.

# Appendix A: scripted build

`bigip/apply.py` builds everything in chapters 5 to 9 through the BIG-IP's REST interface. It uses only the Python standard library, creates what is missing, updates what exists, and can be run again safely.

```
export BIGIP_HOST=<bigip-mgmt> BIGIP_USER=admin BIGIP_PASS='<password>'
export LAB_HOST=<lab-host> ICAP_AV_HOST=<av-host>
export VIP_IP=<vip> INNER_VS_IP=<inner-ip>
export LAB_ROUTE_GW=<gateway>      # only if a lab host is not on a BIG-IP VLAN
RESTART_ASM=1 python3 bigip/apply.py
```

`RESTART_ASM=1` lets the script restart the Application Security service when the antivirus server address changes. Without it the script prints a reminder and leaves the restart to you.

**You should see** the run end with both pools up and no unapplied changes:

```
  policy applied: isModified=False enforcement=blocking active=True
  pool_csv_validator_icap <lab-host>:1345: state=up
  pool_csv_sink <lab-host>:8081: state=up
done
```

`python3 bigip/teardown.py` removes everything the script created, and clears the antivirus server setting.

The password is read from the environment and is never written to disk.

# Appendix B: object inventory

Everything is in partition `CSV` except the last row.

| Object | Name | Purpose |
|---|---|---|
| Routes | `csv_lab_host`, `csv_lab_icap_av` | Reach lab hosts that are not on a BIG-IP VLAN |
| Certificate and key | `csv_lab` | Server certificate for `<vip>` |
| Client SSL profile | `clientssl_csv_lab` | Terminates HTTPS |
| HTTP profile | `http_csv_lab` | Adds `X-Forwarded-For` |
| Monitors | `mon_csv_validator`, `mon_csv_sink` | Health of the two services |
| Pools | `pool_csv_validator_icap`, `pool_csv_sink` | The validator's ICAP port; the application |
| Security policy | `waf_csv_lab` | The WAF rules, the antivirus switch |
| ICAP profile | `icap_csv_validator` | How to call the validator |
| Internal virtual server | `ivs_csv_validator` | Target of request adapt |
| Request Adapt profile | `reqadapt_csv_lab` | Sends requests for a verdict; resets if the validator is down |
| Inner virtual server | `vs_csv_inner` | Validator hop, then the application. On no VLAN |
| LTM policy | `ltm_csv_lab` | Turns on the WAF, then forwards to the inner virtual server |
| Outer virtual server | `vs_csv_lab` | What clients connect to |
| Anti-Virus Protection server | (device-wide) | Where the WAF sends files for scanning |

# Appendix C: what the validator checks

The checks run in this order. The first failure decides the answer.

| Check | Rule | Reason code |
|---|---|---|
| Request | Only `POST /upload`. A declared length of at most 1,310,720 bytes. No chunked transfer | `NOT_FOUND`, `METHOD_NOT_ALLOWED`, `REQUEST_LENGTH_REQUIRED`, `REQUEST_TOO_LARGE` |
| Form | A well-formed `multipart/form-data` body with exactly one part, named `file`, carrying a file | `NOT_MULTIPART`, `MALFORMED_MULTIPART`, `UNEXPECTED_PART`, `MISSING_FILE_FIELD` |
| File name | Plain ASCII; no path; letters, digits, _ and `-`, then one lower-case `.csv`. The encoded form `filename*=` is refused | `FILENAME_ENCODING`, `FILENAME_PATH`, `FILENAME_EXTENSION`, `FILENAME_MISSING` |
| Declared type | `text/csv`, `application/csv` or `application/vnd.ms-excel` | `CONTENT_TYPE` |
| Size | 1 byte to 1,048,576 bytes | `EMPTY_FILE`, `FILE_TOO_LARGE` |
| Encoding | Valid UTF-8 with no NUL bytes | `INVALID_UTF8` |
| Delimiter | Comma, semicolon or tab, whichever the header line uses. At least two columns and two records | `CSV_UNSUPPORTED_DELIMITER`, `CSV_AMBIGUOUS_DELIMITER`, `CSV_TOO_FEW_RECORDS` |
| Quoting | RFC 4180: quotes only around a whole field, doubled inside it, and closed | `CSV_BAD_QUOTING`, `CSV_UNCLOSED_QUOTE` |
| Shape | Every record has as many fields as the header. No blank lines | `CSV_FIELD_COUNT`, `CSV_BLANK_LINE`, `CSV_BAD_LINE_ENDING` |
| Cross-check | A second, independent CSV parser must read the same records | `CSV_PARSER_DISAGREEMENT` |
| Formulas and markup | See below | `CSV_FORMULA_CELL`, `CSV_ACTIVE_MARKUP`, `CSV_ACTIVE_CONTENT` |

## The formula rule

Spreadsheet programs run a cell that begins with `=`, `+`, `-` or `@` as a formula. The validator removes leading spaces, tabs and quote characters from each cell, and from each line of a cell that spans lines, and then refuses the file if what is left begins with one of those characters.

| Cell | Result | Why |
|---|---|---|
| `=1+1`, `@SUM(A1)` | Refused | Begins with `=` or `@` |
| `-5`, `+3.20`, `-.5` | Accepted | A plain signed number |
| `-2+3`, `+cmd...` | Refused | Begins with a sign and is not a plain number |

| Cell | Result | Why |
|---|---|---|
| `555-0100`, `(555) 010-0100` | Accepted | Does not begin with a trigger character |
| `+1 555 0100` | **Refused** | Begins with + and is not a plain number. A known limitation: quote such values differently, or relax the rule |
| three spaces then `=1` | Refused | Leading white space is ignored first |
| `"=1+1"` | Refused | Quoting a field does not make it safe |
| a quoted cell whose second line is `=1+1` | Refused | Every line of a cell is checked |
| `<script>`, `<!DOCTYPE`, `</p>` | Refused | Markup |
| `a < b` | Accepted | Not a tag |
| `javascript:`, `vbscript:`, `data:text/html` | Refused | A link that runs code |

Passing these checks does not make a file safe to open in every program. It means the file obeys these rules.

# Appendix D: things to know about BIG-IP 21.1.0.2

These were found while building and testing the lab.

- **Antivirus server changes need a restart** of the Application Security service. Chapter 7.
- **The antivirus server is one setting for the whole BIG-IP.**
- **Request adapt runs before the WAF** when both are on one virtual server. Chapter 8.
- **Request adapt fails open by default.** Chapter 8.
- **An uploaded file's name is a parameter called `filename`.** Its regular expression needs the **Enable** box, and its violation is off by default. Chapter 6.
- **Response codes are enforced.** A code the policy does not list is replaced by the block page. Chapter 6.
- **The built-in executable check recognises Windows (PE), Linux (ELF) and macOS (Mach-O) programs only.** Scripts such as `.sh` and `.bat` are not detected by it. In this lab the file name rule, the validator and the antivirus cover them.
- **Antivirus scanning does not check that a file is a CSV**, and CSV validation does not look for malware. The lab needs both.

# Appendix E: references

- K64356849: Configuring BIG-IP ASM to block file uploads containing binary executable content
- K000138456: The Disallow File Upload of Executables feature does not block script files
- K78925560: Configuring BIG-IP ASM to restrict file uploads based on file extensions
- K000151407: How to test anti-virus (ICAP) integration with BIG-IP ASM/Advanced WAF
- K16221101: iRules LX limitations
- BIG-IP 21.1 documentation: *Configuring General ASM System Options*, external anti-virus protection
- RFC 3507: Internet Content Adaptation Protocol (ICAP)
- RFC 4180: Common Format and MIME Type for CSV Files